The Fabius Function in Lean

A Guided Walkthrough of the `ProveIt` Formalization

Structure, dependencies, proof architecture, exact computation, Fourier analysis, asymptotics, and approximation theory

Prepared from the Lean 4 development in

`Analysis/FabiusFunction`

Repository snapshot inspected 24 August 2026

Lean 4.32.0; mathlib pinned by the repository manifest

This document is an explanatory companion to the formal source. It is not a substitute for the kernel-checked proofs.

Abstract

The Fabius function is a classical example of a smooth function with an elementary-looking self-similarity law and unexpectedly rich arithmetic, Fourier-analytic, probabilistic, and asymptotic structure. Vladimir Reshetnikov's `ProveIt` repository contains a large Lean 4 development that formalizes both of Juan Arias de Reyna's 2017 papers on the function, proves the existence and uniqueness of a canonical bounded solution, constructs its signed global extension, verifies exact evaluation algorithms at dyadic rationals, develops moment and denominator theory, reconstructs Rvachev's Fourier characterization, and goes substantially beyond the papers through q-binomial identities, discrete-limit theorems, sharp saddle-point asymptotics, and Legendre expansions.

This document is a proof-oriented walkthrough of that codebase. Its aim is not to restate every declaration in source order. Instead, it explains why the development is split into its present modules, which dependencies are mathematically essential, which are merely packaging dependencies, and how the principal proofs are made acceptable to Lean. Particular attention is paid to the two parallel foundations of the project—exact rational arithmetic and analysis under an abstract `IsFabius` specification—and to the bridge that identifies them. The discussion also records source corrections that become unavoidable in a formal proof, recurring proof-engineering idioms, recommended reading paths, and practical advice for extending the library.

Reader's map

A mathematically oriented reader should begin with Chapters 2, 5, 7, 9, and 14. A Lean developer should read Chapters 1, 3, 4, 20, and 21. Readers interested in the two source papers can use the theorem map in Appendix B; readers trying to locate a particular implementation should use the annotated module guide in Appendix C.

Scope and conventions

The repository is active source code, and module boundaries can evolve. Names and dependencies described here refer to the state of the `main` branch inspected on 24 August 2026. The walkthrough quotes identifiers and very short fragments where useful, but it paraphrases proof bodies rather than reproducing them. The exact Lean source remains the authoritative statement of hypotheses, namespaces, coercions, and imported lemmas.

Mathematically, two functions must be kept distinct throughout:

- the bounded cumulative-distribution form $F\colon \mathbb{R} \to [0,1]$, equal to 0 on the left of the unit interval and to 1 on the right; and
- the compactly supported even Rvachev up-function up, obtained by folding or differentiating the bounded form and naturally living near $[-1,1]$.

The code also defines a signed global extension of the bounded function. It is this extension, rather than the clamped bounded map, that satisfies certain translation and dyadic identities for arbitrary real arguments. Several apparent contradictions in informal accounts disappear once these three objects are not conflated.

Lean names are displayed in a monospaced face, for example `Fabius.IsFabius`, `Fabius.fabiusDyadicValue`, and `FabiusSharpAsymptotic.lean`. Mathematical equations are written in conventional notation even when the formal theorem uses a more elaborated type, a subtype coercion, or a filter formulation.

Contents

Chapter 1

Building and entering the development

1.1 Repository location and toolchain

The relevant source tree is

`Analysis/FabiusFunction/Lean/`

and the umbrella import is `FabiusFunction.lean`. The Lake package declares a Lean library named `FabiusFunction` whose source directory is the path above. At the inspected snapshot, `lean-toolchain` selects `leanprover/lean4:v4.32.0`; `lake-manifest.json` pins a corresponding mathlib revision and the transitive packages used by mathlib, including Aesop, Batteries, Qq, proof widgets, and the import-graph tooling.

The normal build command from the project root is:

```
lake build FabiusFunction
```

A useful first test file is:

```
import FabiusFunction

open Fabius

#check IsFabius
#check fabius_spec
#check rvachevUp
#check fabiusDyadicValue
#check extendedFabius
```

Importing the umbrella module is convenient for exploration. During development, importing a narrow leaf module is preferable: it shortens rebuilds, makes accidental dependencies visible, and helps preserve the intended layering.

1.2 What the umbrella module exposes

The top-level `FabiusFunction.lean` does not contain the development itself. It is an import façade. Its principal imports collect four broad products:

1. the formalizations of the two 2017 papers, through `Paper05442.lean` and `Paper06487.lean`;
2. the independent asymptotic project, through `PaperFabiusAsymptotic.lean`;
3. the audit and repair of a k-fold Thue–Morse draft, through `PaperKFoldThueMorse.lean`; and
4. later analytic and approximation developments, including negative-Laplace periodic corrections, Mellin finite parts, translated Legendre series, and least-squares optimality.

This is a deliberate public surface. The umbrella import answers the question "what has the repository established?" It is not the best answer to "where is this theorem proved?" For the latter, one should descend into the paper aggregators or the subject-specific module families.

1.3 A practical exploration workflow

A productive workflow in an editor with the Lean language server is:

1. open the theorem used by a paper aggregator;
2. inspect its type before its proof;
3. use "go to definition" on the first nontrivial helper;
4. note whether the helper belongs to the exact-arithmetic or analytic side of the project;
5. follow imports upward only until the mathematical mechanism becomes clear; and
6. return to the wrapper theorem to see how casts, side conditions, and notation are discharged.

The wrapper theorem is often intentionally short. The real content may be a sequence of modules: one establishing a recurrence over $\mathbb{N}$ or $\mathbb{Q}$, one proving an analytic recurrence for an arbitrary object satisfying `IsFabius`, and one proving that both recurrences share the same initial values.

Architectural point

The development is easier to understand as a directed acyclic graph than as a directory listing. Exact arithmetic starts without the canonical analytic function. Generic analysis starts from an abstract specification rather than from a constructed term. Only after both sides are mature does the code identify exact rational expressions with analytic values and package a canonical function.

Chapter 2

The mathematical objects and their Lean interfaces

2.1 The bounded function

The foundational module `Basic.lean` introduces the closed unit interval as a subtype and uses it as the codomain of a bounded map. Schematically, the first layer has the shape

```
abbrev UnitInterval := Set.Icc (0 : Real) 1
abbrev BoundedFabius := Real -> UnitInterval
```

The subtype codomain records the range bound in the type itself. The projection to $\mathbb{R}$, which the code exposes through a convenience definition such as `fabiusReal`, is used for calculus and integration. This division of labor avoids reproving $0 \le F(x) \le 1$ every time a measure-theoretic bound is required, while still giving the differentiable real-valued function expected by mathlib's analysis APIs.

The abstract predicate `IsFabius` packages the characteristic properties of a candidate. Its fields express, in essence:

$$\begin{aligned} F(x) &= 0 && (x \le 0),\\ F(x) &= 1 && (1 \le x),\\ F(x) + F(1-x) &= 1 && (0 \le x \le 1),\\ F'(x) &= 2F(2x) && (0 \le x \le \tfrac{1}{2}), \end{aligned}$$

together with global smoothness. The precise formal structure uses pointwise derivative predicates and interval membership. Later lemmas extend the symmetry law from the core interval to a statement valid for all real x, with tail cases handled separately.

2.1.1 Why specify before constructing?

Most analytic theorems are stated for a variable F with hypothesis `hF : IsFabiusF`. This is an important architectural choice. It allows derivative identities, Taylor formulas, moment recurrences, and dyadic uniqueness to be proved without unfolding the fixed-point construction. The existence module can then instantiate this generic API, while uniqueness is proved once at the level of the specification.

This organization resembles an interface-and-implementation pattern:

- `Basic.lean` supplies the interface;
- `Differential.lean`, `DyadicAnalytic.lean`, and related modules prove generic consequences of the interface;
- `Existence.lean` supplies the implementation; and
- the canonical term and its theorem `fabius_spec` let later files forget the implementation again.

2.2 The Rvachev up-function

The compactly supported function up is the symmetric "twin" of the bounded cumulative form. The code defines `rvachevUp` by folding the bounded function around the center of the unit interval. Up to the chosen normalization and translation, the relation is the familiar one

$$\mathrm{up}(x) = F(x+1) \quad (-1 \le x \le 0), \qquad \mathrm{up}(x) = 1 - F(x) \quad (0 \le x \le 1),$$

with zero outside the support. The exact Lean definition is arranged to make symmetry and support proofs tractable and to match the Fourier normalization used in the original paper.

The up-function is the more natural object for Fourier analysis because it is even, smooth, compactly supported, and normalized to have integral one. The bounded form is the more natural object for dyadic arithmetic because its derivative equation and boundary values generate finite Taylor recurrences.

2.3 The signed global extension

The bounded function cannot satisfy an unrestricted translation law: it is constant on both tails. The repository therefore defines `extendedFabius`, a signed global extension assembled as a locally finite sum of translated copies of up, weighted by the Thue–Morse sign. In informal notation it has the form

$$\widetilde{F}(x) = \sum_{k \in \mathbb{Z}} \varepsilon(k)\ \mathrm{up}(x-k),$$

where only finitely many summands are nonzero at any given x. Lean nevertheless treats the expression through a `tsum`, so the library must prove local finiteness, summability, and the reduction of the infinite sum to a finite set before elementary rearrangements are legal.

On the unit interval the extension agrees with the bounded function. Away from that interval it carries the signs and period-two behavior needed for global dyadic formulas. This distinction is used pervasively in `GlobalExtension.lean`, `GlobalDyadic.lean`, and the q-binomial modules.

Formalization-sensitive point

When an informal statement writes $F(x)$ for all real x, inspect the theorem's target. It may refer to the bounded map, the real projection of that map, the compactly supported up-function, or the signed extension. Several corrected statements in the repository amount precisely to choosing the right one.

2.4 Auxiliary analytic objects

`Basic.lean` also reserves names for the analytic transforms that later modules identify:

- a complex sinc function with a removable value at zero;
- the Fourier transform and infinite sinc product associated with up;
- moment and half-moment generating series;
- real and complex versions of $(\exp z - 1)/z$, again with the singularity removed; and
- integral definitions of moments over the support.

Defining these objects early stabilizes names and normalizations. Proofs of convergence, equality, and differentiability are deferred to modules with the necessary imports. This keeps `Basic.lean` conceptually small even though it determines the vocabulary of the whole project.

Chapter 3

The dependency architecture

3.1 A layered view

The following diagram suppresses many helper modules but captures the principal flow. Arrows point from prerequisites to consumers.

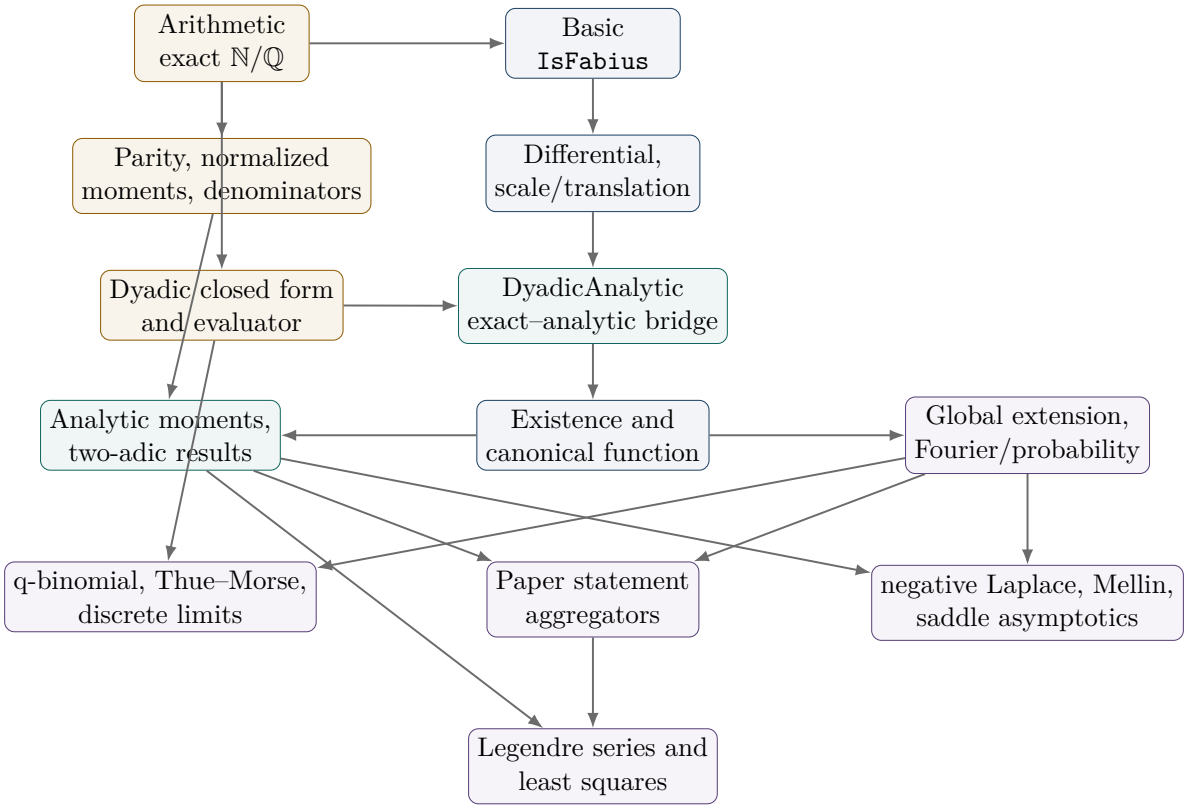

Figure 3.1: Conceptual dependency graph. The source contains finer-grained helper modules between nearly every displayed pair.

3.2 Why existence is not the first analytic file

A newcomer might expect `Existence.lean` to precede every theorem about the function. Instead, the code first proves a great deal about an arbitrary `IsFabius` object. This is not circular. The direction is:

1. assume a candidate satisfies the abstract laws;
2. derive its dyadic values and prove that any two candidates agree on a dense set;
3. construct one candidate by a contraction mapping; and
4. invoke the already-proved generic uniqueness theorem.

This order isolates the hard analytic construction from the algebraic uniqueness mechanism. It also means that the dyadic evaluator is not merely a computational afterthought: it participates in the proof that the canonical object is unique.

3.3 Two independent uniqueness arguments

The repository contains two conceptually different uniqueness proofs.

The first is adapted to the bounded specification. The derivative and symmetry equations determine exact values at every dyadic rational. Dyadics are dense, and the candidates are continuous, so agreement on dyadics implies agreement everywhere.

The second is adapted to Rvachev's original compactly supported characterization. The derivative/refinement law is transformed into a Fourier-domain sinc equation. Iteration forces the entire Fourier transform to equal an infinite sinc product. Fourier injectivity on an appropriate smooth rapidly decreasing class then forces the original functions to coincide.

Keeping both arguments is valuable. The dyadic proof is computational and feeds exact arithmetic; the Fourier proof mirrors the historical characterization and explains the infinite product.

3.4 Paper aggregators versus theorem-producing modules

Files named `Paper05442.lean`, `Paper06487.lean`, and `PaperFabiusAsymptotic.lean` are indexes and audit records. They import theorem-producing modules and expose a source-facing collection of results. For example, `PaperStatements.lean` gives declarations whose numbering and prose correspond to the arithmetic paper, but the actual proof of a denominator theorem may be distributed across `NormalizedMoments.lean`, `Parity.lean`, `TwoAdic.lean`, and `DenominatorBound.lean`.

This separation has three advantages:

- source numbering does not dictate internal dependency boundaries;
- corrected hypotheses can be documented at the aggregator without contaminating low-level lemmas; and
- later developments can reuse strong internal theorems rather than wrappers tailored to a paper's notation.

Chapter 4

The exact-arithmetic foundation

4.1 Why the development begins over $\mathbb{N}$ and $\mathbb{Q}$

The module `Arithmetic.lean` is deliberately independent of the canonical analytic function. It defines the integer and rational sequences that later turn out to be moments, dyadic values, normalizing factors, and denominator bounds. This has a profound practical advantage: exact identities can be proved using induction, finite sums, divisibility, and polynomial normalization without carrying measurability, differentiability, or convergence hypotheses.

The exact side is not just a transcription of closed formulas. It supplies executable definitions. One can evaluate a dyadic rational by normalization and recursion, and Lean's kernel can reduce the resulting rational expression. The same definitions then serve as specifications for correctness theorems.

4.2 Binary weight and the Thue–Morse sign

The smallest combinatorial ingredients are the binary digit sum and its sign:

$$s_2(n) = \text{number of ones in the binary expansion of } n, \qquad \varepsilon(n) = (-1)^{s_2(n)}.$$

In the code these appear under names such as `binaryWeight` and `thueMorseSign`. Elementary identities under doubling and successor are proved early because they drive both the dyadic evaluator and the q-binomial development:

$$s_2(2n) = s_2(n), \qquad s_2(2n+1) = s_2(n) + 1,$$

so that $\varepsilon(2n) = \varepsilon(n)$ and $\varepsilon(2n+1) = -\varepsilon(n)$.

Lean's natural-number division and remainder operations require explicit control of positivity. Proofs that are a one-line observation on binary expansions often become a short cluster of lemmas about `Nat.div`, `Nat.mod`, powers of two, and bit constructors. Establishing this API once prevents the higher-level files from repeatedly descending to bit arithmetic.

4.3 Product normalizations

Several finite products recur in moment denominators. The arithmetic module names them rather than expanding them at every use. Principal examples include variants of

$$\prod_{j=1}^{n}(2^j-1), \qquad \prod_{j=1}^{n}(2^{2j}-1), \qquad (2n-1)!!.$$

The source uses names such as `mersenneProduct`, `evenMersenneProduct`, and `oddDoubleFactorial`. Their basic positivity, nonvanishing, splitting, and divisibility properties are the denominator-management toolkit for later proofs.

A formal proof benefits from keeping two versions of a formula:

- a mathematically transparent quotient in $\mathbb{Q}$; and
- a division-free equality in $\mathbb{N}$, obtained after multiplying by a common product.

The quotient is convenient for stating a moment recurrence. The division-free equality is the robust object for induction, parity, and divisibility, because it avoids repeated side goals asserting that denominators are nonzero.

4.4 Moment and half-moment recurrences

The exact sequences `moment` and `halfMoment` are defined recursively as rational numbers. They model, respectively, moments of the centered up-function and half-interval moments of the bounded function. The definitions mirror the recurrences obtained analytically by integration by parts and scaling.

Alongside them, `momentNumerator` and `halfMomentNumerator` are natural-number recurrences with the common denominators cleared. Their role is easy to underestimate. They provide:

1. a finite, executable representation of exact values;
2. a target for strong-induction proofs of closed forms;
3. a clean setting for parity arguments; and
4. a bridge to reduced rational numerators and denominators.

A recurring proof pattern is to start from the rational recurrence, multiply by the anticipated global normalizing product, distribute that product through a finite sum, and use factorial or Mersenne product splitting to identify every term with an earlier natural numerator. Once the normalizations are designed correctly, the final goal is a polynomial identity in natural numbers and powers of two.

Lean pattern

For exact rational identities, the development usually proves all denominator factors positive or nonzero near the beginning of the proof, rewrites divisions into a common denominator, and invokes `field_simp` only after the relevant nonzero facts are available. The remaining equality is discharged by a combination of finite-sum rewrites, `ring_nf`, and induction hypotheses. Calling `field_simp` too early tends to produce enormous and poorly shaped goals.

4.5 Closed dyadic formulas

The exact values at inverse powers of two are represented by a sequence such as `halfMoment FabiusValue`. A more general closed expression, exposed through a name such as `fabiusDyadic`, gives the value at $a/2^n$ as a finite sum involving the inverse-power table and powers determined by the local Taylor formula.

The formalization maintains a distinction between:

- a closed formula suited to mathematical theorems;
- a Horner form suited to evaluation;
- a unit-interval evaluator that clamps or reflects inputs; and
- a global evaluator that incorporates Thue–Morse signs and period-two reduction.

This separation makes correctness modular. One first proves that the Horner program computes the finite polynomial, then that recursive bit reduction computes the closed formula, then that the closed formula equals the analytic function.

4.6 The inverse-power table

The function `fabiusInversePowTwoTable` computes a prefix of exact values

$$F(2^{-1}), F(2^{-2}), \ldots, F(2^{-n}).$$

A table-based implementation avoids recomputing the same recurrence at every recursive step. Correctness is expressed through prefix stability: extending a table preserves all previously computed entries. This is essential because the evaluator may request values in an order dictated by the bits of the numerator rather than by a single monotone loop.

Formal prefix stability is typically a theorem about list lookup or finite-indexed entries. It combines:

- length calculations;
- bounds for `Fin` indices or `List.get?`;
- compatibility of a recurrence with appending a new term; and
- extensional equality of the old prefix.

Such lemmas look implementation-specific, but they prevent computational code from leaking into the mathematical correctness theorem.

4.7 Horner evaluation

The local Taylor polynomial is evaluated by `fabiusTaylorHorner`. Horner's rule is not only computationally efficient; it is proof-friendly. The correctness theorem is an induction over the coefficient list, and the induction invariant is a suffix polynomial. In contrast, a direct power-sum implementation forces repeated reasoning about exponents, list indices, and truncated ranges.

The coefficients are rational and involve powers of two coming from iterated differentiation. The proof therefore includes cast-normalization lemmas ensuring that a natural exponent or binomial coefficient has the same interpretation in $\mathbb{N}$, $\mathbb{Q}$, and $\mathbb{R}$. The code frequently uses explicit type annotations at precisely these interfaces; their apparent verbosity documents where coercion inference would otherwise be ambiguous.

4.8 Recursive removal of the highest bit

At the center of the evaluator is a structurally terminating function such as `fabiusDyadicUnitAux`. Given a dyadic numerator a and exponent n, it removes the highest set bit of a. Write

$$a = 2^b + r, \qquad 0 \le r < 2^b.$$

A Taylor identity expresses the value at $a/2^n$ using the value and inverse-power data associated with the smaller remainder r. Since $r < a$ whenever $a > 0$, the recursive call decreases in a well-founded order.

This is the computational counterpart of traversing the ones in the binary expansion of a. The number of recursive Taylor reductions is therefore controlled by $s_2(a)$, not by the magnitude of a itself. Together with precomputation of the inverse-power table, the design yields the approximate complexity described in the repository documentation: a quadratic table phase in the exponent and a bit-weight-dependent evaluation phase.

Lean does not accept "the highest bit was removed" as a termination argument. The source proves a chain of inequalities connecting the bit logarithm, the largest power of two not exceeding a, and the remainder. This chain is packaged so the recursive definition can use a measure. The correctness proof later reuses the very same decomposition, an example of an implementation lemma doing double duty as a mathematical induction lemma.

4.9 Clamping, reflection, and the global evaluator

The unit-interval wrapper `fabiusDyadicUnit` normalizes a dyadic representative and applies the endpoint and symmetry laws. The rational-valued `fabiusDyadicValue` accepts an integer numerator and power-of-two denominator, managing negative values, inputs beyond one, and equivalent unreduced representations.

For the signed extension, `extendedFabiusDyadicValue` first reduces the integer part modulo the relevant period and then applies a Thue–Morse sign. The exact theorem is stronger than a formula restricted to reduced representatives: multiplying numerator and denominator by a common power of two leaves the output unchanged. This refinement invariance is proved explicitly rather than delegated to quotient types, because the evaluator's input is computational data, not an abstract element of a dyadic localization.

Proof idea

There are three distinct notions of normalization:

1. arithmetic normalization of a fraction by canceling powers of two;
2. geometric normalization into $[0, 1]$ using tails and reflection; and
3. global normalization modulo translation, with a Thue–Morse sign.

The code proves each normalization preserves the intended mathematical value before composing them.

4.10 Recognizing dyadic rationals

The definitions `IsDyadicRational` and `dyadicExponent?` support a user-facing exact evaluator for rational inputs. A rational is dyadic when its reduced denominator is a power of two. The decision procedure must connect:

- the internal normalized numerator and positive denominator of $\mathbb{Q}$;
- a Boolean or optional power-of-two test; and
- a proposition asserting the existence of an integer numerator and natural exponent.

The proof that recognition is sound is separate from the proof that the returned exponent and numerator evaluate correctly. This makes failure meaningful: a non-dyadic rational is rejected because no representation with a power-of-two denominator exists, not merely because a search limit was reached.

4.11 Arithmetic API design lessons

The exact core illustrates several design principles that recur throughout the repository.

First, executable functions and theorem-friendly formulas need not be identical definitions. They are connected by named correctness lemmas. Second, normalization data should be made explicit, especially when a mathematical quotient hides representation choices. Third, the strongest useful invariant is often not the final equality but a stability statement about prefixes, refinements, or common denominators. Finally, natural-number recurrences are frequently the right interface for arithmetic theorems even when the motivating quantities are rational.

Chapter 5

Constructing the canonical bounded function

5.1 The ambient complete metric space

The construction in `Existence.lean` uses Banach's fixed-point theorem. Let

$$I = [0, 1], \qquad C = C(I, \mathbb{R})$$

with the uniform metric. Since the codomain is complete and I is compact, the continuous-map space is complete. The code then cuts out a closed subset of C consisting of functions satisfying two elementary conditions:

$$0 \le f(x) \le 1,$$
$$f(x) + f(1 - x) = 1.$$

The source calls the reflection operator and the admissibility predicate by local names and proves that the admissible set is closed. A linear function, essentially $x \mapsto x$, witnesses nonemptiness.

Using a closed subspace rather than the entire continuous-function space is decisive. Symmetry is not recovered after the fixed point; it is an invariant of the contraction. This makes the integral normalization automatic and improves the contraction constant from the naive value one to one half.

5.2 Extending an interval function

An element of $C(I, \mathbb{R})$ accepts a subtype argument. To integrate it over an ordinary real interval, the code defines an extension by projecting an arbitrary real number to I. This projection clamps values below zero to zero and values above one to one. Continuity follows from continuity of the metric projection to a closed convex interval.

For an admissible f, define the cumulative integral

$$A_f(y) = \int_0^y \widehat{f}(t)\ \mathrm{d}t,$$

where $\widehat{f}$ is the clamped extension. Symmetry implies

$$\int_0^1 f(t)\ \mathrm{d}t = \frac{1}{2}.$$

The Lean proof reflects the integral by the substitution $t \mapsto 1 - t$, adds the two copies, and uses the pointwise symmetry equation. The subtype coercions and interval maps are handled in helper lemmas so the final normalization proof resembles the paper calculation.

5.3 The nonlinear-looking transform that is actually affine

The fixed-point transform is defined piecewise at the midpoint:

$$(Tf)(x) = \begin{cases} A_f(2x), & 0 \le x \le \frac{1}{2}, \\ 1 - A_f(2 - 2x), & \frac{1}{2} \le x \le 1. \end{cases}$$

The two branches agree at $x = 1/2$ because $A_f(1) = 1/2$. Each branch is continuous, and the gluing lemma supplies continuity of Tf on the whole interval. Bounds follow because f is bounded between zero and one; symmetry follows by exchanging the two branches.

The transform is affine on the admissible set. The constants in the right branch cancel when two transformed functions are subtracted. This is why the contraction estimate can be proved through integrals of $f - g$.

5.4 The contraction estimate

Let $h = f - g$ for two admissible functions. Symmetry gives

$$\int_0^1 h(t)\, \mathrm{d}t = 0.$$

For $0 \le y \le 1$, the prefix integral may be estimated in two ways:

$$\left|\int_0^y h\right| \le y\|h\|_\infty, \qquad \left|\int_0^y h\right| = \left|\int_y^1 h\right| \le (1 - y)\|h\|_\infty.$$

Therefore

$$\left|\int_0^y h\right| \le \min(y, 1 - y)\|h\|_\infty \le \frac{1}{2}\|h\|_\infty.$$

This is the key insight of the construction. A crude estimate would give constant one and no contraction. The zero-total-integral identity lets the proof select the shorter of the prefix and complementary suffix.

Lean pattern

The formal estimate is assembled from interval-integral norm bounds rather than from an informal "length times supremum" principle. The code proves integrability of the extended difference, obtains a pointwise uniform bound from the distance in the continuous-map space, applies the interval integral inequality on both subintervals, and rewrites the total integral as zero. A final case split on $y \le 1/2$ selects the useful bound.

Banach's theorem now yields a unique fixed point in the admissible subspace. The source extracts its underlying continuous map and extends it to a bounded real function with constant tails.

5.5 From an integral fixed point to the differential law

A fixed point satisfies an integral equation. On the left half,

$$F(x) = \int_0^{2x} F(t)\ \mathrm{d}t.$$

The fundamental theorem of calculus and the chain rule give

$$F'(x) = 2F(2x).$$

On the right half, differentiation of the reflected branch yields the corresponding equation through symmetry. The proof handles the midpoint and endpoints separately because standard derivative rules apply most directly on open neighborhoods, while the desired structure requests derivatives at boundary points as well.

The source uses `HasDerivAt` statements whenever possible. Such statements compose cleanly through affine maps and integral primitives; a theorem about `deriv` alone would require differentiability side conditions each time it is rewritten. Endpoint derivatives are obtained by showing that the formulas from adjacent pieces agree, using the tail values and symmetry.

5.6 Bootstrapping smoothness

The fixed-point theorem initially gives only continuity. The integral equation gives one derivative. The derivative is a scaled composition of the original continuous function, so it is continuous. Iterating this observation upgrades the function to every finite differentiability order.

The formal proof is organized around reusable scale-and-derivative lemmas rather than a single enormous induction on `ContDiff`. At stage $n+1$, the derivative formula expresses the new derivative in terms of a scaled version of the n-th derivative. Boundary compatibility follows from vanishing on the tails and symmetry. The final theorem packages all finite stages as C^∞.

5.7 Uniqueness by dyadic density

The contraction theorem already gives uniqueness inside the admissible continuous space. The exported specification, however, is phrased for real functions satisfying `IsFabius`; it is useful to prove uniqueness at this abstract level. The later dyadic analysis shows that every such function has the same exact value at every dyadic point. Since dyadics are dense in $\mathbb{R}$ and both candidates are continuous, the functions coincide.

The proof has the standard topological shape: approximate an arbitrary x by a sequence of dyadics, rewrite equality at each sequence term, and pass to the limit by continuity. The nontrivial work lies earlier, in proving exact dyadic equality without assuming the fixed-point implementation.

Architectural point

The fixed-point construction proves that the interface is inhabited. The dyadic theory proves that the interface is subsingleton. This yields a canonical object whose public properties are independent of the construction details.

Chapter 6

Differential identities and the signed global extension

6.1 The role of `Differential.lean`

`Differential.lean` is the calculus engine for an arbitrary `IsFabius` candidate. Starting from the first derivative law on the left half and the globalized symmetry law, it derives corresponding formulas on other intervals and iterates the derivative relation.

The characteristic coefficient after n differentiations is a triangular power of two. In a typical region where all scaled arguments remain in the appropriate branch, one obtains

$$F^{(n)}(x) = 2^{1+2+\cdots+n} F(2^n x) = 2^{n(n+1)/2} F(2^n x).$$

The exact theorem includes domain conditions ensuring that each intermediate scaled point lies in the branch where the derivative law is valid. Reflection introduces signs and translated arguments on the other half.

These iterated formulas explain three later phenomena at once:

- finite Taylor formulas at dyadic points;
- flatness at the support boundary; and
- extremely rapid decay near zero.

6.2 Domain bookkeeping

An informal derivation often differentiates the relation $F'(x) = 2F(2x)$ without restating its domain. Lean forces the domain to be propagated. If a theorem is iterated n times, the proof must establish that $2^j x$ belongs to the relevant interval for every $j < n$. The source packages this as inequalities involving powers of two and dyadic cells.

This bookkeeping is not bureaucratic. One of the asymptotic draft corrections later in the repository arises because a transformed delay-differential identity is valid only beyond a threshold. The formal derivative theorem makes the threshold impossible to omit.

6.3 Scale and translation lemmas

`ScaleTranslation.lean` collects chain-rule identities for functions of the form

$$x \longmapsto c\,F(ax+b), \qquad x \longmapsto c\ \mathrm{up}(ax+b).$$

These are used by approximation arguments, global extension formulas, Fourier transforms, and splines. The value of a dedicated module is that every later proof can invoke a theorem with all factors of a, signs, and differentiability hypotheses already correct.

A common Lean mistake is to rewrite an iterated derivative of a composition as though the affine map contributed only one factor of a. The library's lemmas state the full a^n factor. Specialized powers-of-two corollaries then combine this with the triangular coefficient from the Fabius differential law.

6.4 Local finiteness of the translate sum

`GlobalExtension.lean` proves that the signed translate series defining $\widetilde{F}$ is harmless. Because up has compact support, only finitely many integers k can satisfy $x-k \in [-1,1]$ for fixed x. More strongly, on a compact neighborhood of x, a single finite set of translates contains every nonzero summand.

That stronger local statement is what permits termwise continuity and differentiation. The proof turns support inequalities into integer bounds, places all possible indices in a finite interval of $\mathbb{Z}$, and proves every index outside that set contributes zero. The `tsum` can then be rewritten as a finite sum using a summability-by-finite-support lemma.

6.5 Agreement and global identities

Once local finiteness is available, the signed extension is shown to agree with the bounded function on $[0,1]$. The support of up reduces the sum to one or two adjacent translates, and the defining fold relation simplifies the result.

Translation by an integer changes the Thue–Morse coefficient in a controlled way. Translation by two gives a period-two relation; translation by one introduces a sign or complementary term depending on the normalization. These identities are the setting in which unrestricted versions of the dyadic recurrence are true. `GlobalDyadic.lean` combines them with exact values to remove assumptions that a numerator be reduced or lie between zero and its denominator.

6.6 Why use a `tsum` for a locally finite expression?

A hand proof would define the sum over the two possible integers adjacent to x. The repository instead uses a conceptually uniform sum over all integers. This pays off in translation arguments: shifting x corresponds to reindexing the same sum, and standard infinite-sum equivalences can be reused. The initial cost is a local-finiteness API; the later algebra becomes much cleaner.

Chapter 7

The exact–analytic dyadic bridge

7.1 The purpose of `DyadicAnalytic.lean`

The exact evaluator is useful only after it is tied to the analytic function. `DyadicAnalytic.lean` is the central bridge. It proves, for every `IsFabius` candidate, the same finite recurrences that were defined independently in `Arithmetic.lean`; then it identifies the two by induction.

The proof has four main stages:

1. derive a finite Taylor identity on a dyadic cell;
2. specialize it to inverse powers of two and obtain an endpoint recurrence;
3. prove that the analytic inverse-power sequence equals the exact rational table; and
4. lift this equality to arbitrary dyadic numerators through the highest-bit recursion.

The surrounding modules `DyadicClosedForm.lean` and `DyadicCorrectness.lean` isolate the algebraic and algorithmic portions of this chain.

7.2 Finite Taylor expansion on a dyadic scale

Fix a sufficiently small dyadic base point. The iterated derivative formula expresses every derivative up to a chosen order through a scaled value of F. At the order where the scaled argument leaves the active interval and enters the zero tail, higher derivatives vanish. Taylor's theorem therefore terminates exactly rather than producing an infinite series with a remainder.

A representative identity has the shape

$$F\left(\frac{a+r}{2^n}\right) = \sum_{k=0}^{K} \frac{2^{k(k+1)/2}}{k!} F\left(\frac{a}{2^{n-k}}\right) \left(\frac{r}{2^n}\right)^k,$$

with the base point, displacement, and upper index chosen to match a dyadic cell. The source uses a carefully normalized variant so the coefficients coincide with the exact Horner evaluator.

The proof does not invoke analyticity. It uses a finite-order Taylor theorem with a remainder, then proves the next derivative vanishes on the relevant segment. This is crucial: the Fabius function is smooth but nowhere analytic in the appropriate sense, so an argument based on convergence of an infinite Taylor series would be false.

Formalization-sensitive point

Finite Taylor identities are local consequences of compact support and the differential equation. They do not imply that the function equals its infinite Taylor series in a neighborhood. The formalization later turns the same exact truncation phenomenon into a proof of nonanalyticity.

7.3 Inverse-power values

For a candidate F, define informally

$$A_n(F) = F(2^{-n}).$$

The dyadic Taylor identity at a suitable endpoint gives a recurrence for $A_{n+1}(F)$ in terms of earlier $A_j(F)$. Boundary compatibility and symmetry supply the initial value $A_1(F) = 1/2$.

Independently, `Arithmetic.lean` defines a rational sequence satisfying the same recurrence. Strong induction proves

$$A_n(F) = A_n^{\text{exact}}$$

for every n. The induction step is predominantly algebraic, but the statement lives in $\mathbb{R}$; the proof must cast exact rational sums and products into reals. The source localizes these casts in lemmas asserting that the rational recurrence is preserved by the canonical embedding $\mathbb{Q} \hookrightarrow \mathbb{R}$.

7.4 From closed polynomial to Horner program

Once the analytic value is a finite polynomial in exact inverse-power values, `DyadicAnalytic.lean` invokes the algebraic theorem that the closed polynomial equals `fabiusTaylorHorner`. This is a clean interface point:

- analysis proves the value is the mathematical polynomial;
- arithmetic proves the polynomial has exact rational coefficients; and
- algorithmics proves Horner evaluation computes that polynomial.

No proof about derivatives needs to know how a coefficient list is stored, and no proof about lists needs to know the fundamental theorem of calculus.

7.5 Correctness of highest-bit reduction

Suppose the numerator decomposes as $a = 2^b + r$. The analytic Taylor theorem proves exactly the recurrence used by the recursive evaluator. The recursive call has numerator $r < a$, so strong induction on a applies. At the end, the theorem states that evaluating the exact program and embedding its rational output into $\mathbb{R}$ gives $F(a/2^n)$.

The proof handles endpoint and out-of-range cases before entering the recursion. This matches the program structure and avoids forcing the induction hypothesis to cover impossible branch conditions. Each branch ends with a theorem about the corresponding normalization operation: left-tail clamping, right-tail clamping, reflection, highest-bit Taylor reduction, or refinement.

7.6 Representation independence

The same dyadic rational has infinitely many representations:

$$\frac{a}{2^n} = \frac{2a}{2^{n+1}} = \frac{4a}{2^{n+2}} = \cdots .$$

The evaluator's mathematical specification therefore includes refinement invariance. `DyadicCorrectness.lean` proves that one common factor of two can be introduced or removed without changing the result; induction yields invariance under any common power.

There are two possible proof routes. One can prove both representations equal the analytic function and conclude by transitivity, or prove refinement directly from the exact recursion. The repository contains strong exact correctness results so that the executable layer remains trustworthy even when used independently of analysis. Analytic equality then gives a second conceptual explanation.

7.7 Dyadic density and uniqueness

Let F and G satisfy `IsFabius`. Correctness gives

$$F(q) = G(q)$$

for every dyadic rational q. To extend this to an arbitrary real x, choose dyadic approximants such as

$$q_n = 2^{-n} \lfloor 2^n x \rfloor .$$

Then $q_n \to x$. Continuity gives

$$F(x) = \lim_n F(q_n) = \lim_n G(q_n) = G(x).$$

In Lean, density can be applied through a generic theorem saying continuous maps equal on a dense set are equal, or through an explicit approximating sequence. The repository's dyadic infrastructure supplies the density statement in the exact form required by the function-extensionality proof. Tail values allow the argument to focus on $[0, 1]$ when convenient.

7.8 The global dyadic theorem

`GlobalDyadic.lean` transfers exact evaluation from the bounded interval to `extendedFabius`. The result accepts unrestricted integer numerators and nonreduced dyadic representations. It combines:

1. arithmetic decomposition into an integer translate and a unit-interval remainder;
2. the translation law for the signed extension;
3. exact unit-interval correctness; and
4. representation invariance.

This stronger formulation is the one required by the q-binomial identities, where summation indices naturally produce numerators outside $[0, 2^n]$.

Architectural point

The dyadic bridge is the repository's main "commuting square." One path starts with an analytic function and derives a finite Taylor recurrence; the other starts with exact rational recurrences and an evaluator. The correctness theorem says both paths produce the same real number.

Chapter 8

Moments, generating functions, and exact denominators

8.1 Analytic moments of the up-function

`AnalyticMoments.lean` develops integral interpretations of the exact moment sequences. Because up is smooth and compactly supported, every polynomial moment exists:

$$M_n = \int_{\mathbb{R}} x^n \,\mathrm{up}(x)\, \mathrm{d}x.$$

Evenness immediately gives $M_{2k+1} = 0$. The total mass $M_0 = 1$ is proved from the bounded differential equation and the fundamental theorem of calculus, rather than assumed as part of the definition.

The main recurrence is obtained by integration by parts and a dyadic change of variables. The precise coefficients depend on the chosen centering of up, but the structure is a finite convolution of lower moments with powers of two and binomial coefficients. The same recurrence was used to define `moment` in the exact layer.

8.1.1 Integration over compact support

Mathlib's integral on $\mathbb{R}$ is convenient for Fourier transforms, while interval integrals are convenient for integration by parts. The proof repeatedly converts between them using support lemmas. A standard sequence is:

1. prove the integrand vanishes outside a compact interval;
2. rewrite the whole-line integral as an integral over that interval;
3. apply an interval integration-by-parts theorem;
4. show endpoint terms vanish by support or flatness; and
5. transform the remaining integral by an affine substitution.

The support theorem is therefore not merely qualitative metadata; it is an active rewrite rule used throughout the analytic stack.

8.2 Identifying analytic and exact moments

After deriving the recurrence, the module proves by strong induction that

$$M_n = \text{moment}(n)$$

after embedding the rational value into $\mathbb{R}$. The displayed formula intentionally suppresses namespace and cast syntax; the Lean theorem states them explicitly.

The base case is total mass. In the induction step, the analytic recurrence is rewritten term by term using the induction hypotheses. A cast lemma turns the finite rational sum into the corresponding real sum. The exact recurrence then closes the goal.

The same method identifies half-interval moments with `halfMoment`. The source-facing theorem starts at positive index, while `halfMoment_eq_integral_formula_all` packages the normalized zeroth value and the integral cases into one statement for every natural index. These moments are connected more directly to inverse-power Fabius values, so they become the analytic bridge between moment theory and the dyadic table.

8.3 Generating functions

The moment generating function is represented both as an integral and as a power series. Compact support gives uniform bounds

$$|M_n| \le CR^n,$$

which are more than sufficient for convergence of

$$\sum_{n\ge 0} M_n \frac{z^n}{n!}$$

for every complex z. Dominated convergence justifies interchanging sum and integral, giving the usual identity with $\int e^{zx}\,\mathrm{up}(x)\,\mathrm{d}x$.

`MomentPowerSeries.lean` develops the power-series side in a reusable form. It links the exact inverse-power table, moments, and entire functions. The code uses mathlib's `FormalMultilinearSeries` and `HasFPowerSeriesAt` machinery only where it pays off; many coefficient identities are easier as direct statements about `HasSum` and factorially weighted series.

8.4 Removable singularities

Expressions such as

$$\frac{e^z - 1}{z}$$

appear in half-moment generating functions. The source defines a total function with value one at zero, then proves continuity and analyticity across the removable singularity. This avoids carrying $z \neq 0$ through every downstream identity.

The proof combines the exponential power series with division by z term by term. In Lean, defining the value at zero first and proving a global power-series expansion is often cleaner than starting with a piecewise quotient and repeatedly rewriting the nonzero branch.

8.5 Normalized half moments

`NormalizedMoments.lean` proves a closed denominator normalization of the form

$$\text{halfMoment}(n) = \frac{H_n}{(n+1)!\prod_{j=1}^{n}(2^j-1)},$$

where H_n is the division-free natural numerator defined in `Arithmetic.lean`. The theorem is not obtained by asking Lean to normalize a large rational expression automatically. It is proved by strong induction with a deliberately chosen common denominator.

At the induction step, every earlier half moment has a denominator whose factorial and Mersenne products are prefixes of the target denominator. The missing suffix factors are inserted explicitly. Product-splitting lemmas show that multiplying an earlier term by its suffix gives the target common denominator. The natural numerator recurrence was designed to be exactly the resulting cleared equation.

8.6 Normalized even moments

`NormalizedEvenMoments.lean` proves the analogous expression for even centered moments. A representative denominator consists of an odd double factorial and an even-indexed Mersenne product:

$$M_{2n} = \frac{E_n}{(2n+1)!!\prod_{j=1}^{n}(2^{2j}-1)}.$$

The actual indexing in the source is chosen to align with its recursive sequence, but the structural point is the same. Odd moments vanish analytically, while even moments carry all arithmetic information.

The proof requires more combinatorial normalization than the half-moment case because binomial coefficients and parity restrictions appear in the recurrence. Factorial identities convert binomial coefficients into products compatible with the odd double factorial. The final division-free recurrence is again a natural-number equality.

8.7 Parity through Lucas' theorem

`Parity.lean` proves that the natural moment numerators are odd. The key combinatorial input is Lucas' theorem modulo two: a binomial coefficient $\binom{n}{k}$ is odd exactly when every one-bit of k occurs at a one-bit position of n. Consequently the number of odd entries in row n of Pascal's triangle is

$$2^{s_2(n)}.$$

The moment recurrence is reduced modulo two. Most terms vanish in pairs or carry explicit factors of two; the surviving binomial pattern is counted using Lucas' theorem. The source must bridge several representations of a finite range—natural inequalities, `Fin` types, and `Fin set.range`—before applying mathlib's Lucas results. Dedicated cardinality lemmas hide this bridge from the final parity induction.

8.8 Reduced numerators, denominators, and two-adic valuation

`TwoAdic.lean` turns the normalization and parity theorems into exact assertions about reduced rational numerators and denominators. If a displayed quotient has odd numerator and a denominator whose power of two is known, then no cancellation of two-adic factors is possible. This determines the two-adic valuation of the reduced denominator.

The argument differs for odd and even indices. Odd-index statements reduce to the normalized even moments. Even-index statements use the half-moment recurrence together with the Lucas count of odd binomial coefficients. The formal proof includes lemmas relating:

- the numerator and denominator fields of a normalized rational;
- divisibility by two in $\mathbb{N}$ and $\mathbb{Z}$;
- coprimality in reduced form; and
- a valuation statement for the dyadic value $F(2^{-n})$.

8.9 Denominator bounds

`DenominatorBound.lean` and `HalfMomentDenominator.lean` assemble divisibility results into explicit bounds. Rather than attempting to compute the reduced denominator directly, the proof supplies a common denominator known to clear every term in a recurrence. The reduced denominator then divides that common denominator by the universal property of rational normalization.

This method is robust under changes of indexing and does not require unique factorization calculations for the full denominator. Exact two-adic theorems can be layered on top where a sharper result is needed.

8.10 Bernoulli recurrences and source-facing arithmetic

`BernoulliRecurrences.lean` translates some moment identities into Bernoulli-number language used by the arithmetic paper. The formalization keeps Bernoulli manipulations separate from the basic moment recurrence: the latter is elementary and executable, while the former uses a larger special-function and finite-sum API.

`PaperStatements.lean` then exposes propositions and theorems with numbering matching the paper. Its proofs are often short compositions of the stronger internal results described above. This is a good place to learn what each source theorem says, but not always the place to learn why it is true.

Chapter 9

Fourier analysis and Rvachev's original characterization

9.1 Why the up-function is the Fourier object

The bounded Fabius function has nonzero constant tails and is not integrable on the line. The Rvachev up-function is compactly supported, smooth, and even, so it belongs to every L^p space and, after viewing it as a smooth function with compact support, defines a Schwartz function. This is the natural object for the Fourier-transform theorems in `FourierAnalytic.lean` and `OriginalUniqueness.lean`.

The original characterization asks for a smooth compactly supported function satisfying a refinement or derivative equation and a normalization. The repository formalizes both existence and uniqueness. The uniqueness proof is especially instructive because it turns a differential equation in physical space into a multiplicative recurrence in frequency space.

The source predicate retains the paper's explicit assumption that its dilation scale is positive. The smart constructor `IsOriginalFabius.mk_of_derivative_law` shows that this sign is actually forced by the remaining hypotheses: a mean-value point in $(-1, 0)$ reduces the derivative law to $k\varphi(2c + 1) = 1$ with a strictly positive second factor.

The two formal solution spaces are connected explicitly. The theorem `IsFabius.isOriginalFabius_rvachevUp` says that the Rvachev fold of every bounded Fabius solution satisfies the original compact-support characterization at scale two. Conversely, `isOriginalFabius_iff_existsUnique_isFabius` says that every original solution has scale two and is the fold of a unique bounded Fabius solution. This is the correct tail-free equivalence: a direct predicate iff for an arbitrary bounded function would be false because its fold never inspects values on $(1, \infty)$. The theorem `rvachevUp_eq_iff_eqOn_Iic_one` makes that loss of information exact: two folds agree if and only if their candidates agree on $(-\infty, 1]$. Accordingly, `isFabius_iff_isOriginalFabius_rvachevUp_and_rightTail` restores the sharp fixed-candidate iff by adjoining only the condition $F(x) = 1$ for $x > 1$.

9.2 Fourier conventions

Formal Fourier proofs are extremely sensitive to normalization. Mathlib's transform includes a fixed kernel convention; the source therefore writes explicit factors of π, 2π, and the imaginary

unit rather than silently adopting a paper convention. The sinc factor is defined compatibly with that convention and extended continuously at zero.

A schematic transform identity is

$$\widehat{\mathrm{up}}(z) = \mathrm{sinc}(\pi z)\,\widehat{\mathrm{up}}(z/2).$$

Depending on whether the source theorem uses angular or ordinary frequency, the argument of sinc is adjusted. The Lean theorem's constants should be taken as authoritative; the important structural feature is multiplication by one sinc factor and halving of frequency.

9.3 Transforming the derivative equation

The source first proves that a candidate satisfying the original assumptions is compactly supported and smooth enough to be represented as a Schwartz map. It then applies the Fourier derivative theorem and the affine change-of-variable theorem. A derivative contributes a frequency factor; a dilation contributes both a reciprocal Jacobian and a rescaled frequency; translations contribute complex phases.

After simplification, the phases combine into a sine quotient, producing the sinc recurrence. The mass normalization gives

$$\widehat{\mathrm{up}}(0) = \int_{\mathbb{R}} \mathrm{up}(x)\,\mathrm{d}x = 1.$$

This is the seed value that controls the limit after infinitely many refinements.

Lean pattern

The proof does not rewrite Fourier integrals by hand at every step. It first establishes membership in the function class required by mathlib, then invokes library theorems for derivative, translation, and dilation. Only the final algebra of complex exponentials is expanded. This concentrates the fragile measure-theoretic assumptions near the beginning of the proof.

9.4 Iteration to a finite product

Iterating the refinement identity n times gives

$$\widehat{\mathrm{up}}(z) = \left(\prod_{j=0}^{n-1} \mathrm{sinc}\left(\frac{\pi z}{2^j}\right)\right) \widehat{\mathrm{up}}\left(\frac{z}{2^n}\right).$$

This is proved by induction. The Lean induction must reconcile two equivalent ways of writing a finite product: appending the last factor versus splitting off the first. A small library of `Finset.range` product identities keeps the main theorem readable.

As $n \to \infty$, $z/2^n \to 0$, so continuity of the Fourier transform gives

$$\widehat{\mathrm{up}}(z/2^n) \to 1.$$

The finite products converge to an infinite product because $1 - \mathrm{sinc}(w) = O(w^2)$ near zero and $\sum_j |z|^2/4^j$ converges.

9.5 The infinite sinc product

`FourierProduct.lean` develops the technical product independently. It proves convergence, continuity in the frequency variable, and compatibility between the finite products produced by iteration and the chosen definition of `rvachevFourierProduct`.

Infinite products in Lean can be represented through partial products and limits rather than through a single primitive operator. The source proves that the sequence is Cauchy using a summable bound on deviations from one. Special care is taken at zeros of a sinc factor: logarithmic product arguments are invalid there, whereas direct partial-product convergence remains sound.

The result is the celebrated formula

$$\widehat{\mathrm{up}}(z) = \prod_{j\geq 0} \operatorname{sinc}\left(\frac{\pi z}{2^j}\right)$$

with the indexing adjusted to the normalization used by the code.

9.6 Fourier injectivity and uniqueness

If two original candidates satisfy the same hypotheses, the iteration argument gives the same Fourier transform for both. The source then uses injectivity of the Fourier transform on Schwartz functions to conclude equality. The final transition is:

1. package each smooth compactly supported function as a Schwartz map;
2. prove their transforms are equal as functions;
3. apply Schwartz-space Fourier injectivity; and
4. project the equality back to ordinary functions.

The theorem also shows that the scaling constant in the original functional equation is forced, rather than being an arbitrary parameter. Evaluation at zero or comparison of total mass leaves only the canonical value, which in the repository's normalization is two.

9.7 Moment series from the Fourier product

Expanding the transform at zero recovers moments. The logarithm of the sinc product yields cumulant-like sums, while direct multiplication gives a power series whose coefficients are rational combinations of powers of two. `FourierAnalytic.lean` relates this analytic expansion to `rvachevMomentSeries` and the exact moment sequence.

The proof again prefers an identity of entire functions followed by coefficient comparison. Establishing an entire power series permits differentiation at zero to extract moments without repeatedly justifying differentiation under the integral sign.

9.8 Poisson summation

The original paper's Poisson-summation theorem is formalized in a dedicated module. Compact support and smoothness place the function in the Schwartz class, so mathlib's Poisson summation framework applies. The main work is normalization: the theorem from the library and the paper use different Fourier conventions and summation coordinates. The proof rewrites the transform through the sinc product and then simplifies the sampled factors.

This is representative of source-facing modules: the analytic theorem already exists in a reusable library form, while the paper theorem is a carefully normalized corollary.

Proof idea

The Fourier uniqueness proof is a rigidity argument. The differential/refinement equation repeatedly pushes all unknown information into the value of the transform at a frequency tending to zero. Normalization fixes that limiting value, so no freedom remains.

Chapter 10

Finite convolutions, probability, and approximation

10.1 The probabilistic representation

The infinite sinc product is also the characteristic function of an infinite sum of independent centered random variables whose scales decrease geometrically. A typical model is

$$X = \sum_{j \geq 1} 2^{-j} U_j,$$

where the law of each U_j is chosen so its characteristic function is the corresponding sinc factor. The exact formulation in `ProbabilityRepresentation.lean` matches the convolution construction used in the original paper.

The code first treats finite sums. Their laws are finite convolutions of simple compactly supported probability measures. Characteristic functions multiply, so the finite convolution transform is the finite sinc product. Tightness and almost-sure or distributional convergence then identify the limit law with the up-function density.

10.2 Finite convolution measures

A formal convolution proof must verify that every measure involved is finite, usually a probability measure. The source defines the elementary measures, proves their support and total mass, and then uses induction to show the convolution remains a probability measure. Associativity is applied only after the necessary sigma-finiteness or finiteness instances are in scope.

The support of the n-fold convolution is the Minkowski sum of the elementary supports. Geometric decay keeps these supports in a fixed compact interval. This uniform support bound supplies tightness essentially for free and gives uniform moment bounds.

10.3 Weak convergence

`WeakConvergence.lean` proves convergence against bounded continuous test functions. There are two complementary mechanisms:

- convergence of random partial sums, followed by bounded convergence; and
- convergence of characteristic functions plus identification of the unique limit.

The repository packages the result in the measure-theoretic language needed by later interval and distribution-function arguments.

The test-function formulation avoids trying to prove pointwise convergence of densities too early. Weak convergence is stable under convolution and directly reflects the probabilistic construction. To recover the Fabius function, the code later specializes to approximations of interval indicators.

10.4 Step approximants

`StepApproximationLimit.lean` studies the explicit step functions arising from finite convolution coefficients. The coefficient arrays satisfy symmetry and unimodality properties. These imply monotonicity of the approximants on each side of the center and provide pointwise upper and lower controls.

The desired limit is continuous, but indicator functions of intervals are not continuous test functions. The source uses an interval-average squeeze. Integrals over a small neighborhood converge by weak convergence; monotonicity bounds the point value of a step function between nearby interval averages. Shrinking the neighborhood and using continuity of the limit gives pointwise convergence.

10.5 The endpoint half-weight correction

Closed interval indicators double-count shared endpoints when adjacent steps are assembled. In ordinary integration this affects a measure-zero set, but a pointwise theorem about the approximating function sees the discrepancy. The formalization therefore uses a representative with half weight at step boundaries.

This correction is a good example of Lean revealing a genuine mathematical issue rather than a mere elaboration problem. The source paper's almost-everywhere intuition is adequate for an integral identity, but not for the stated pointwise approximation. Once the half-endpoint convention is adopted, neighboring intervals partition correctly and the convergence theorem has the intended endpoint values.

10.6 Coefficient unimodality

The finite convolution coefficients form rows with strong symmetry and unimodality. The proof proceeds through a recurrence: the next row is obtained by averaging shifted blocks of the previous row. Symmetry is immediate from the symmetric elementary measure; monotonicity requires comparing overlapping partial sums.

Lean handles the row as a finite function or list. The proof establishes index bounds before rewriting recurrence entries. Boundary entries, where one shifted block leaves the valid range, are split into separate cases. These technical cases explain why the coefficient lemmas occupy a substantial part of `StepApproximationLimit.lean`: the final analytic squeeze is

comparatively short once monotonicity is available.

10.7 From distribution functions to the bounded Fabius function

The limiting probability law has density up. Its cumulative distribution, after the repository's translation convention, is the bounded Fabius function. The proof integrates the density over a half-line, uses support to reduce to a compact interval, and applies the fold relation between F and up.

This gives an independent conceptual construction of the same object as the Banach fixed point. In the formal dependency graph it is normally developed after the canonical function is available, because identification is easier than rebuilding the entire `IsFabius` interface from measure convergence. Mathematically, however, it explains why the function is a distribution function.

10.8 Exact finite Taylor formulas and nonanalyticity

`OriginalPaperSupplement.lean` packages further theorems from the original paper. At centered dyadic points, sufficiently high derivatives vanish on one side because repeated scaling reaches the support boundary. Taylor's theorem becomes an exact finite polynomial identity on a small interval.

If the function were analytic at such a point, its Taylor series would terminate and the function would be a polynomial in a neighborhood. The differential equation and support properties rule this out unless the function were trivial. Thus the exact local truncation contributes to a nowhere-analyticity theorem.

The formal proof distinguishes several notions:

- `ContDiff` smoothness;
- existence of derivatives of all orders at a point;
- equality to the Taylor series on a neighborhood; and
- mathlib's analytic-at predicate.

A proof that merely observes all derivatives vanish at a boundary point establishes flatness, not by itself nonanalyticity everywhere. The module supplies the additional translation and dyadic-density argument needed for the global conclusion.

10.9 Corrections recorded by the original-paper aggregator

`Paper05442.lean` and its supplement explicitly document several source-sensitive points. Among them are:

- finite convolution must be stated at the finite stage rather than with an already limiting object;
- step-function endpoints require half weights for pointwise identities;

- a factor of the transform variable is needed in one displayed Fourier equation;
- a formula involving a positive order requires the hypothesis $n > 0$; and
- a global dyadic identity needs the signed extension or an adjusted normalization.

These are not hidden as comments in low-level proofs. The aggregator treats them as part of the formal audit of the paper.

Chapter 11

The two paper formalizations as navigable APIs

11.1 The first paper: `Paper05442.lean`

The module named after arXiv:1702.05442 imports the original uniqueness proof, probability representation, weak convergence, step approximation, Poisson summation, and supplementary source theorems. Its purpose is to certify coverage of the paper as a whole.

A useful reading order is:

1. `OriginalCharacterization.lean` and `OriginalUniqueness.lean` for the defining characterization and the equivalence of solution spaces;
2. `ProbabilityRepresentation.lean` for the random/convolution model;
3. `WeakConvergence.lean` for the measure limit;
4. `StepApproximationLimit.lean` for pointwise approximation;
5. `PoissonSummation.lean` for the sampled Fourier identity; and
6. `OriginalPaperSupplement.lean` for moments, Taylor formulas, rationality, and nonanalyticity.

The aggregator's theorem names are source-facing, but the mathematical objects remain those of the common core. This avoids a second, paper-specific definition of the function.

11.2 The arithmetic paper: `Paper06487.lean`

The second aggregator, named after arXiv:1702.06487, imports `Paper06487Supplement.lean`, which in turn draws on `PaperStatements.lean` and the arithmetic/analytic bridge modules. It covers the paper's propositions on moments, exact dyadic values, denominators, parity, and two-adic valuations.

The internal proof structure is more extensive than the paper numbering suggests:

$$\text{exact recurrence} \longrightarrow \text{normalized numerator} \longrightarrow \text{parity/divisibility} \longrightarrow \text{reduced rational theorem.}$$

Each arrow is represented by one or more modules. The final paper theorem is often a cast or notation wrapper around an internal theorem with a stronger domain or a more explicit divisibility conclusion.

11.3 Statements that remain questions or conjectures

A formalization should not turn an open question into a theorem. `PaperStatements.lean` records the paper's Question 5, Definition 12, and Conjecture 16 as definitions or propositions expressing the claimed predicate, not as proofs of the unresolved assertion. This is part of the source-coverage discipline: every numbered item is represented, but its logical status is preserved.

The surrounding proved theorems may establish initial cases, necessary conditions, or denominator bounds relevant to the conjecture. Their names make clear when the result is a theorem about the conjectural predicate rather than a proof of the conjecture itself.

11.4 Why a separate coverage document matters

The repository's `PAPER_COVERAGE.md` maps source theorem numbers to Lean declarations. This is more than project management. It helps distinguish three questions:

1. Has the mathematical claim been formalized?
2. Where is the strongest internal theorem proved?
3. Which wrapper corresponds to the source numbering and notation?

Appendix B reproduces the navigational content of that map in compact form and adds the surrounding module families.

Chapter 12

Thue–Morse and q-binomial identities

12.1 Why q-binomial algebra appears

The Thue–Morse sign converts binary digit structure into alternating sums. When a finite Taylor formula is applied repeatedly across dyadic cells, the resulting coefficients naturally involve Gaussian, or q-binomial, coefficients at $q = 1/2$. The repository develops this relationship in a series of modules ranging from exact finite identities to global analytic series.

The core finite identity has the flavor

$$\sum_{k=0}^{2^n-1} \varepsilon(k)(x+k)^m,$$

which vanishes for low degree and factors in a controlled way at the critical degree. q-binomial coefficients provide a compact description after binary decomposition. The Fabius values enter when the polynomial identity is inserted into a scaled finite Taylor expansion.

12.2 The finite q-Pochhammer infrastructure

`HalfQBinomial.lean` develops exact q-binomial algebra at $q = 1/2$ over $\mathbb{Q}$. The finite q-Pochhammer product is represented without analytic convergence issues:

$$(a;q)_n = \prod_{j=0}^{n-1}(1 - aq^j).$$

At $q = 1/2$, clearing powers of two turns many factors into Mersenne numbers. This is why the Mersenne products introduced in `Arithmetic.lean` reappear.

The module defines a rational `halfQBinomial` and proves its q-Pascal recurrence. It then proves a finite q-binomial theorem by induction. The induction step is not a black-box appeal to an imported polynomial theorem; it matches the precise normalization required downstream. Summands are shifted and paired so the q-Pascal recurrence telescopes into the next product.

12.3 Why prove a specialized q-binomial theorem?

Mathlib contains general polynomial and combinatorial infrastructure, but a specialized theorem at $q = 1/2$ gives the repository:

- exact rational coefficients with predictable denominators;
- direct compatibility with existing Mersenne products;
- a recurrence usable by kernel computation; and
- statements in the same finite-sum form as the Fabius formulas.

A general theorem would still require a substantial specialization layer. Proving the specialized identity directly keeps casts and nonvanishing side conditions under control.

12.4 Raw, scalar, and polished formulas

The q-binomial family is intentionally split into several modules:

- `FabiusRawQBinomialFormula.lean` and `FabiusRawQBinomialScalar.lean` establish algebraic sums before all Fabius-specific simplifications;
- `FabiusQBinomialFormula.lean` proves the exact finite identity in its principal form;
- `FabiusQBinomialScalarFormula.lean` specializes coefficient-valued statements to scalar values; and
- `FabiusDyadicQBinomialScalar.lean` inserts the exact dyadic evaluator.

This decomposition protects the core polynomial identity from unnecessary analytic dependencies. It also allows translation invariance to be proved at the polynomial level and reused after evaluation.

12.5 The unrestricted finite identity

A notable strength of the formal result is that it is not restricted to a reduced dyadic representative or to $0 \le m \le 2^n$. It is stated for arbitrary natural parameters in the range supported by the algebraic expression. The signed global extension absorbs the translation behavior that would otherwise force an artificial range hypothesis.

The proof proceeds by induction on the binary depth. Splitting a range into even and odd indices transforms the Thue–Morse sign into a difference of two scaled copies. The induction hypothesis applies to each copy, and the q-Pascal recurrence combines the coefficients. Polynomial degree bounds control vanishing terms.

Proof idea

The even/odd split is the common engine behind Thue–Morse cancellation and q-binomial recursion. In Lean, the decisive step is a reindexing theorem that expresses a sum over `Finset.range(2*N)` as sums over $2k$ and $2k + 1$. Once that lemma has the right shape, the sign and power transformations simplify automatically.

12.6 Translation invariance as a polynomial theorem

One of the strongest internal statements says that the relevant finite q-expression is independent of a translation parameter because it is a polynomial of degree below the cancellation threshold. This is proved before specializing the parameter to real or complex values. As a result, common translation invariance holds over a broad coefficient ring and later applies uniformly to $\mathbb{R}$ and $\mathbb{C}$.

This is a recurring proof-design strategy: prove a fact at the most algebraic level where it is true, then transport it through evaluation homomorphisms. Analytic specializations become one-line consequences of polynomial extensionality.

12.7 Taylor expansion in the q parameter

`FabiusQBinomialTaylor.lean` analyzes the finite expression as a polynomial or power series in q. Coefficients are finite and exact; no convergence theorem is needed. The Taylor expansion supplies estimates for the later discrete-limit theorem when a complex shift parameter approaches its limiting value.

The source separates coefficient extraction from norm estimates. First it proves an exact coefficient formula. Then it bounds each coefficient by a combinatorial majorant, and finally sums the majorant. This ordering produces reusable algebraic theorems and prevents analytic inequalities from obscuring finite identities.

12.8 The global binary-reduction series

`FabiusBinaryReductionSeries.lean` and `FabiusGlobalQBinomialSeries.lean` pass from finite identities to an infinite series representing the signed extension. The binary reduction follows the same highest-bit philosophy as the exact evaluator, but now the steps are organized as a global q-series.

A subtle endpoint correction places the outer index at $m = 0$, not $m = 1$. At the initial endpoint the missing term is not negligible; it carries the base value that makes the telescoping identity exact. The formal proof reveals this immediately because a finite partial-sum formula leaves a nonzero boundary term.

The convergence argument uses strong coefficient decay inherited from products of powers of two and factorials. Uniform convergence on compact sets permits termwise continuity and, where required, differentiation. The final equality is first proved at dyadic points using exact correctness and then extended by continuity, or derived directly by telescoping finite approximations depending on the module.

12.9 Parity-power series

`FabiusParityPowerSeries.lean` packages a related expansion in which the coefficients are controlled by binary parity. The Thue–Morse sign is represented as an exact coefficient sequence, and finite cancellation gives a high-order zero at the origin. Bounds on the remaining coefficients yield an entire or locally uniform series as appropriate.

This module is a bridge between combinatorics and complex analysis. The coefficient identities are exact, while convergence is proved with norms and comparison tests. Keeping these two layers separate makes it possible to reuse the parity theorem in purely arithmetic settings.

Chapter 13

Discrete limits and complex shifts

13.1 The shape of the limit theorem

The discrete-limit project starts from finite q-binomial/Thue–Morse rows depending on a real location x and a complex parameter q. For nondyadic x, an individual finite row can depend genuinely on q. The theorem is that, after the prescribed normalization and as the row depth tends to infinity, the limit exists and is independent of q, equaling the relevant Fabius expression.

This distinction between finite-stage dependence and limiting independence is explicitly formalized. It prevents an overstrong claim that each approximant is constant in the parameter.

13.2 Uniform spline approximants

`FabiusUniformSpline.lean` introduces centered finite Thue–Morse splines. These are piecewise-polynomial functions whose coefficients encode a finite convolution row. They are uniformly bounded, vanish on the nonpositive half-line, and repeat on the nonnegative axis as signed two-unit blocks. `FabiusComputability.lean` proves the global error estimate

$$\sup_{x\in\mathbb{R}} |S_p(x) - \mathcal{F}(x)| \le 2^{-p} \qquad (p \ge 1).$$

Thus they converge uniformly on all of $\mathbb{R}$, while their bounded-CDF specialization converges uniformly to F on $[0,1]$.

The source proves continuity at knots by comparing adjacent polynomial pieces. Finite-difference cancellation ensures derivatives up to the expected order also match. Rather than relying on a generic spline package, the proof works directly with the explicit pieces because their coefficients must later be transformed into the q-binomial sum.

13.3 Complex-shift control

`FabiusComplexShiftSpline.lean` and `FabiusDiscreteLimitComplexShift.lean` control evaluating a finite spline at a complex displacement. A Taylor expansion separates the real centered value from correction terms. Uniform derivative bounds, together with the scale of the shift, show the corrections vanish in the limit.

The proof is careful not to use order arguments on $\mathbb{C}$. Norm bounds replace real inequalities, and factorial denominators are estimated in $\mathbb{R}$ before being cast into complex norms. The finite nature of the Taylor expansion is again important: each finite spline has bounded degree, so no complex analytic remainder theorem is required.

13.4 The Toeplitz averaging mechanism

`FabiusDiscreteLimitToeplitz.lean` rewrites the normalized row as a Toeplitz-type weighted average of centered spline values. The weights need not be nonnegative: their row sum is one and their total variation is uniformly bounded, while the sampled spline degrees all tend to infinity. These facts transfer convergence of the splines to convergence of the averages.

The formal theorem isolates three hypotheses:

1. a uniform bound on the input sequence;
2. convergence on every fixed central window; and
3. concentration and normalization of the weights.

This abstraction is useful beyond the specific Fabius application and keeps the final limit proof from becoming a nested epsilon argument.

13.5 Integration and identification of the limit

`FabiusDiscreteLimitIntegration.lean` is the final integration layer. Uniform convergence of cumulative distributions or spline primitives permits passage to the limit under an interval integral. Boundary terms telescope into the global binary-reduction series, which has already been identified with `extendedFabius`.

The result is stated for $x \geq 0$ and complex q in the supported domain. The restriction on x reflects the normalization of the finite rows; global signed-extension identities can then translate or reflect additional cases.

13.6 Proof flow of the discrete-limit family

A recommended source order is:

1. `FabiusRawQBinomialFormula.lean`;
2. `FabiusQBinomialFormula.lean`;
3. `FabiusBinaryReductionSeries.lean`;
4. `FabiusUniformSpline.lean`;
5. `FabiusComplexShiftSpline.lean`;
6. `FabiusDiscreteLimitToeplitz.lean`;
7. `FabiusDiscreteLimitComplexShift.lean`; and
8. `FabiusDiscreteLimitIntegration.lean`.

The sequence moves from exact finite algebra, through real approximation, to complex perturbation, averaging, and final analytic identification.

Formalization-sensitive point

When reading these files, distinguish the row index from the polynomial degree, the dyadic scale, and the q-parameter. Informal drafts often use a single letter for more than one role. The Lean development uses separate variables and explicit type annotations, which can make theorem statements longer but prevents illegal substitutions.

Chapter 14

The logarithmic scale near the flat endpoint

14.1 Why ordinary Taylor asymptotics fail

Every derivative of the bounded Fabius function vanishes at zero, yet $F(x) > 0$ for every $x > 0$. Thus no nonzero power of x is the leading term. The correct scale is logarithmic and, at first order, quadratic in $-\log x$.

The asymptotic development introduces

$$\phi(t) = F(2^{-t}), \qquad g(t) = -\log \phi(t).$$

As $x \downarrow 0$, the variable $t = -\log_2 x$ tends to infinity. This turns dyadic scaling into unit translation and is the natural coordinate for the delay equation.

14.2 The exact logarithmic delay identity

`FabiusLogScale.lean` differentiates $\phi(t) = F(2^{-t})$. For $t \geq 1$, the argument 2^{-t} lies in the left branch where the differential equation applies. The chain rule gives

$$\phi'(t) = -(\log 2)2^{-t}F'(2^{-t}) = -2(\log 2)2^{-t}F(2^{1-t}).$$

Since $\phi(t-1) = F(2^{1-t})$, this becomes a delay relation. Rewriting in terms of $g = -\log \phi$ and its derivative produces the exact identity formalized in the module:

$$g(t) - g(t-1) = \log g'(t) - \log(\log 2) - (1-t)\log 2, \qquad t \geq 1.$$

The source first proves positivity of $F(2^{-t})$ and the relevant logarithmic slope, so every logarithm is legal.

Formalization-sensitive point

The threshold $t \geq 1$ is essential. The bounded function's derivative law does not hold on the same branch for all real t. An informal draft treated the delay identity as global; the Lean theorem records the correct domain and downstream asymptotic arguments are formulated with eventual filters.

14.3 Positivity near zero

Taking a logarithm requires strict positivity, not merely nonnegativity. The repository obtains positivity from monotonicity and exact dyadic values. Given $x > 0$, choose a sufficiently large inverse power of two below x. Exact arithmetic shows $F(2^{-n}) > 0$, and monotonicity gives $F(x) \geq F(2^{-n}) > 0$.

This proof is representative of the interaction between exact and analytic layers: a qualitative analytic property at every positive real point is obtained from exact values on a dense cofinal sequence.

14.4 The quadratic logarithmic law

`FabiusLogSquaredAsymptotic.lean` proves

$$\frac{g(t)}{t^2} \longrightarrow \frac{\log 2}{2}.$$

Equivalently,

$$\log F(x) \sim -\frac{(\log(1/x))^2}{2\log 2} \qquad (x \downarrow 0).$$

The proof first establishes sufficiently sharp estimates at natural values of t, where the exact inverse-dyadic recurrence is available. It then extends them to real t by monotonicity and floor/ceiling comparison.

If $n \leq t < n+1$, monotonicity gives

$$g(n) \leq g(t) \leq g(n+1)$$

with the inequality direction determined by the decreasing behavior of $F(2^{-t})$. Dividing by t^2 and using $n/t \to 1$ transfers the natural asymptotic to real arguments.

14.5 A quantitative coarse bound

The same module proves a bound of the form

$$\left| g(t) - \frac{\log 2}{2} t^2 \right| \leq Ct \log t$$

for all sufficiently large t. This is weaker than the eventual sharp formula but strong enough to establish flatness and compare F with elementary scales.

Such a concrete inequality is more useful in Lean than a bare $o(t^2)$ statement. It can be inserted into exponential comparisons with explicit thresholds, and it supports later proofs that every power of x dominates $F(x)$ while every $e^{-c/x}$ is dominated by $F(x)$.

14.6 Filter formulations

The repository states asymptotics both in the $t \to \infty$ coordinate and as $x \to 0^+$. Conversion lemmas relate the filters under $x = 2^{-t}$. A typical proof establishes a limit along `atTop`,

composes with the continuous map $t \mapsto 2^{-t}$, and then identifies the image filter with `nhdsWithin0(Set.Ioi0)`.

Explicit conversion lemmas are preferable to ad hoc substitutions in each theorem. They ensure that positivity domains and logarithm definitions are carried through uniformly.

Chapter 15

Negative Laplace analysis and the periodic correction

15.1 The negative moment generating function

Let

$$G(-s) = \int_{\mathbb{R}} e^{-sx}\ \mathrm{up}_+(x)\ \mathrm{d}x$$

denote the relevant negative Laplace transform in the repository's normalization. The exact moment-generating product gives, for $s > 0$,

$$G(-s) = \prod_{j\geq 1} \frac{1 - e^{-s/2^j}}{s/2^j}.$$

The product's logarithm is

$$L(s) = \sum_{j\geq 1} \left[\log(1 - e^{-s/2^j}) - \log(s/2^j)\right].$$

`NegativeLaplace.lean` constructs this series carefully and proves absolute convergence after grouping terms in a form adapted to small and large arguments.

15.2 The kernel and its bounds

The central scalar kernel is the logarithm of a normalized Bose factor. For small u, the quotient $(1 - e^{-u})/u$ tends to one, so its logarithm is $O(u)$. For large u, the exponentially small correction $\log(1 - e^{-u})$ is controlled by a geometric series.

The source proves elementary inequalities before summing over dyadic scales. Typical estimates use

$$0 \leq -\log(1 - v) \leq \frac{v}{1 - v} \qquad (0 \leq v < 1),$$

with $v = e^{-u}$. The resulting majorants are summable after the dyadic split at the scale where $s/2^j$ crosses one.

15.3 The exact dilation equation

The product shifts cleanly under $s \mapsto 2s$. Removing the first factor and reindexing the rest gives an exact functional equation for $L(s) = \log G(-s)$. Solving the associated difference equation in the logarithmic variable $u = \log_2 s$ produces a quadratic polynomial plus a periodic remainder.

The formal decomposition is

$$L(s) = -\frac{(\log s)^2}{2\log 2} + \frac{1}{2}\log s + R(\log_2 s) + E(s),$$

where $R(u+1) = R(u)$ and the forward-tail error satisfies an explicit exponential bound. A representative bound proved in the module is

$$|E(s)| \le \frac{e^{-s}}{(1-e^{-s})^2},$$

and for s beyond a fixed threshold this is simplified to a constant multiple of e^{-s}.

15.4 How the periodic function is constructed

The periodic term is not guessed from numerics. The code builds it by comparing the exact logarithm with a particular quadratic solution of the dilation equation. The difference is invariant under doubling up to a forward tail. Iterating the dilation toward infinity makes that tail summable; adding its convergent correction yields an exactly invariant function of $\log_2 s$.

Concretely, one introduces a defect

$$D(s) = L(s) + \frac{(\log s)^2}{2\log 2} - \frac{1}{2}\log s.$$

The dilation identity shows that $D(2s) - D(s)$ is an exponentially small kernel. A forward sum of these defects corrects D into a function satisfying $P(2s) = P(s)$. Writing $R(u) = P(2^u)$ gives period one.

Proof idea

The periodic correction is the homogeneous solution of a dyadic difference equation. The quadratic logarithmic terms form a particular solution. Exponentially small forward tails determine which periodic solution corresponds to the actual infinite product.

15.5 Continuity and regularity

`PeriodicCorrection.lean` proves continuity of R. The defining correction is a locally uniformly convergent series on positive compact intervals. The proof covers a compact interval $[a, b] \subset (0, \infty)$, bounds every tail term uniformly by a summable geometric sequence depending on a, and applies a Weierstrass-type theorem.

Later modules strengthen regularity. `PeriodicRegularity.lean` differentiates the correction series term by term after proving uniform convergence of derivative tails. `PeriodicMean.lean` computes or normalizes its mean. `PeriodicFourier.lean` identifies Fourier coefficients through Mellin-transform data.

15.6 Mellin–Bose analysis

`MellinBose.lean` studies the Mellin transform of the Bose kernel that appears in the logarithm. The transform has factors involving the Gamma and zeta functions. Poles encode the polynomial logarithmic terms, while values on the imaginary lattice generated by $2\pi/\log 2$ encode the Fourier coefficients of the period-one correction.

The formal proof proceeds first in a vertical strip where the defining integral is absolutely convergent. It then relates the integral to known Gamma/zeta representations and uses analytic continuation or finite-part identities only in modules that import the required complex-analysis machinery.

15.7 Finite parts and Bromwich preparation

`MellinFinitePart.lean`, `BoseFinitePartIntegral.lean`, and related modules remove singular polynomial pieces from the Mellin inversion formula. A finite-part integral is represented as an ordinary convergent integral after subtracting the local Laurent terms explicitly.

This decomposition is important for formal contour arguments. Instead of manipulating a divergent integral symbolically, Lean sees a sum of:

- explicit residues;
- a convergent integral on a truncated contour; and
- error terms with proved norm bounds.

The periodic Fourier series then emerges from residues at the imaginary lattice.

15.8 Why the periodic term cannot be dropped

The oscillatory correction $R(\log_2 s)$ is bounded but nonconstant. Therefore it is lower order than the quadratic logarithmic term but not an error tending to zero. Any asymptotic formula claiming an $o(1)$ or $O(1/\log s)$ residual after omitting R is false.

`FabiusWikipediaObstruction.lean` and related audit modules formalize this obstruction. They prove nonconstancy through a nonzero Fourier coefficient or a pair of points with unequal values. Sampling along two dyadic phase subsequences then produces distinct limit points for the alleged residual.

15.9 Identification with the generating function

After constructing the product and its logarithm, `NegativeLaplace.lean` proves equality with the analytic generating function defined earlier. The route is:

1. establish the product formula for finite truncations;
2. prove convergence to the infinite product;
3. establish the same functional equation and normalization for the generating function; and

4. use uniqueness or direct limit passage to identify them.

This closes the loop between exact moment series, Fourier/probability products, and the asymptotic Laplace object.

Chapter 16

Bromwich inversion and the sharp saddle point

16.1 From a transform to the endpoint density

The sharp asymptotic of $F(x)$ is obtained by inverting a Laplace transform. Schematically,

$$F(x) = \frac{1}{2\pi \mathrm{i}} \int_{\sigma-\mathrm{i}\infty}^{\sigma+\mathrm{i}\infty} e^{sx}\, H(s)\, \mathrm{d}s,$$

where H is built from the negative-Laplace product and normalization factors. The contour parameter $\sigma > 0$ is chosen at a saddle depending on x.

`BromwichSaddle.lean` and `FabiusBromwichInput.lean` provide the inversion identity and the analytic bounds needed to move to the saddle line. Formal contour integrals are represented through finite vertical segments followed by a limit. Tail bounds justify passage to the infinite line.

16.2 The Lambert phase

Differentiating the dominant exponent produces an equation in which the saddle scale appears both linearly and inside a logarithm. The exact solution is expressed through a lower Lambert-type function. The repository defines the relevant phase $\lambda(x)$ and proves it is the unique positive solution of the saddle equation in the required range.

The advantage of retaining the exact Lambert phase is that all-order expansions can be stated without repeatedly composing truncated asymptotic series. The leading behavior is

$$\lambda(x) = \log \frac{1}{x} + \log\log \frac{1}{x} + O(1),$$

but the exact implicit relation captures the corrections needed in both the exponential and Gaussian normalization.

`FabiusLambertPhase.lean`, `FabiusLambertRates.lean`, and `FabiusLambertSaddle.lean` establish monotonicity, growth, derivative bounds, and the exact saddle equation. Separate modules for small-argument expansions later translate λ into elementary logarithms to any fixed order.

16.3 Exact reduction at the saddle

`FabiusSharpExactReduction.lean` packages the inversion calculation into an exact error decomposition. In logarithmic form, it has the conceptual shape

$$\log F(x) - M(x) = \log(\text{normalized saddle mass}) + \text{forward-tail correction},$$

where $M(x)$ contains the explicit quadratic-logarithmic, Lambert, Gaussian, and periodic pieces.

This theorem is the critical interface between contour analysis and asymptotic algebra. All later approximations can focus on two controlled terms: a normalized central integral tending to one, and an exponentially flat tail inherited from the negative-Laplace product.

16.4 Central and tail decomposition

Write the vertical contour variable as $s = \sigma + \mathrm{i}y$. The integral is split at a central radius $|y| \le Y(\lambda)$:

$$I = I_{\text{central}} + I_{\text{tail}}.$$

Near the saddle, a Taylor expansion of the exponent begins with a negative quadratic term. Away from the saddle, a minor-arc estimate shows strict decay of the real part.

The module family includes `FabiusSaddleCentral.lean`, `FabiusSaddleTail.lean`, `FabiusLambertMinorArc.lean`, and reference-weight modules. The split allows different techniques:

- local cumulant expansions and Gaussian comparison in the center;
- coarse but uniform exponential inequalities in the tail.

Trying to use the local Taylor series on the entire contour would give an unjustified remainder.

16.5 The Gaussian reference weight

After rescaling y by the square root of the second derivative at the saddle, the leading central factor is $e^{-u^2/2}$. `FabiusSaddleReferenceWeight.lean` and `FabiusSaddleReferenceTail.lean` collect exact Gaussian integrals and tail bounds in the normalization used by the contour.

The normalized saddle mass is divided by the Gaussian main integral so its limit should be one. This normalization simplifies logarithms: once the ratio is $1 + O(1/\lambda)$, its logarithm is controlled by a standard $\log(1+u)$ estimate.

16.6 Cumulants and derivative bounds

The logarithm of the negative-Laplace product has derivatives expressible as dyadic sums of kernel derivatives. `LaplaceCumulantAsymptotics.lean`, `FabiusLambertDerivativeBounds.lean`, and related modules prove uniform estimates for these cumulants at the saddle.

The second derivative determines the Gaussian width. Higher derivatives determine correction polynomials. The source proves bounds of the form

$$\kappa_r(\lambda) = O(\lambda^{1-r/2})$$

after the chosen normalization. Exact exponents vary with convention, but the hierarchy guarantees that each additional Taylor order contributes another inverse power of the large phase.

16.7 Minor arcs and tail decay

A complex product may have oscillatory peaks away from zero. `FabiusLambertMinorArc.lean` proves that, outside the central window, enough sinc/Bose factors lose modulus to force decay. The proof divides the frequency axis into ranges:

1. a near-central range where a quadratic cosine or exponential inequality applies;
2. an intermediate range where a fixed block of dyadic factors supplies uniform loss; and
3. a far range controlled by elementary transform decay and integration by parts.

Each range has a separate numerical inequality, making the final integral estimate a finite case split rather than an opaque global bound.

16.8 The first correction

`FabiusFirstSaddleCorrection.lean` computes the first non-Gaussian term. Cubic and quartic derivatives of the phase contribute through Gaussian moments. Odd terms integrate to zero against the even Gaussian; the first surviving coefficient combines the quartic term and the square of the cubic term.

The code represents the correction as a polynomial in the rescaled central variable. Exact Gaussian moments are evaluated recursively or through double factorials. The resulting rational coefficient is then inserted into the normalized saddle mass expansion.

16.9 The corrected sharp formula

`FabiusSharpAsymptotic.lean` states the sharp logarithmic asymptotic. In schematic form,

$$\log F(x) = -\frac{\lambda(x)^2}{2\log 2} + A\lambda(x) + B\log\lambda(x) + R\big(\theta(x)\big) + C + O\left(\frac{1}{\lambda(x)}\right),$$

where $\lambda(x)$ is the exact lower-Lambert phase and R is the nonconstant period-one correction evaluated at an exact phase $\theta(x)$. The constants and phase shifts are fixed by the repository's transform normalization.

Exponentiating gives an asymptotic equivalence for $F(x)$ itself. The proof first bounds the logarithmic error, then uses continuity of the exponential and the fact that $e^{O(1/\lambda)} = 1+O(1/\lambda)$.

16.10 Failure of an uncorrected elementary formula

A simpler formula obtained by dropping the periodic term may still reproduce the leading $(\log x)^2$ scale. It does not have a residual of order $1/(-\log x)$. The repository proves this by

choosing two phase subsequences along which the periodic correction approaches different values. The residual therefore has distinct limit points and cannot tend to zero.

Similarly, a proposed quotient of elementary exponentials is shown to decay on the wrong scale. `FabiusQuotientExponentialMismatch.lean` compares logarithms and proves the quotient is not asymptotically equivalent to F, even if a finite numerical range looks persuasive.

Architectural point

The sharp theorem does not emerge from manipulating the delay equation alone. The delay equation explains the coarse quadratic scale. The periodic term and correct constant require the exact transform product; the Gaussian factor and inverse-phase corrections require Bromwich inversion and saddle analysis.

Chapter 17

All-order asymptotic expansions

17.1 Why the exact phase is retained

An all-order expansion can be expressed either in the exact saddle variable $\lambda(x)$ or directly in powers of $-\log x$ and iterated logarithms. The repository takes the exact-phase expansion as primary. This avoids a common error: composing two finite asymptotic series and claiming an arbitrary-order remainder without proving uniform control of the composition.

The exact-phase theorem has the form

$$\mathcal{M}(x) \sim \sum_{j \geq 0} \frac{c_j}{\lambda(x)^j},$$

where $\mathcal{M}(x)$ is the normalized saddle mass. For every N, the difference from the first N terms is $O(\lambda^{-N})$.

17.2 Formal logarithm of the Laplace product

`FabiusLambertFormalLog.lean` extracts a formal series for the local phase. It treats logarithms, exponentials, and reciprocal series in a truncated polynomial ring. The goal is algebraic: compute coefficients that must occur if a sufficiently smooth analytic remainder is controlled.

Separating formal algebra from analytic remainder estimates is essential. Formal power-series identities can be checked by ring normalization and coefficient extensionality. Their analytic validity is supplied later by bounds on the omitted terms.

17.3 Higher small-argument expansion

`FabiusLambertHigherExpansion.lean` and `FabiusLambertAllOrderSmallArgument.lean` prove the expansion of the exact Lambert phase in terms of

$$L = \log(1/x), \qquad \ell = \log L.$$

For each fixed order, λ is represented by a polynomial in $1/L$ whose coefficients are polynomials in ℓ, plus a rigorously bounded remainder.

The proof uses the implicit saddle equation. An approximate phase is substituted into the equation; formal algebra shows cancellation through the desired order. Monotonicity and a derivative lower bound for the implicit function turn the equation residual into a bound on the phase error.

17.4 Gaussian polynomial contractions

The central exponential has an expansion

$$e^{-u^2/2}\exp\left(\sum_{r\geq 3}\lambda^{1-r/2}a_r(iu)^r\right).$$

Truncating the second exponential produces polynomials in u and $1/\sqrt{\lambda}$. Odd monomials vanish after integration; even monomials contract to Gaussian moments. The modules `GaussianPolynomialContraction.lean`, `GaussianPolynomialWholeIntegral.lean`, and the tail modules formalize this mechanism.

A polynomial is represented by finitely supported coefficients. The contraction functional maps u^{2k} to $(2k-1)!!$ and odd powers to zero. Proving that contraction commutes with finite sums and scalar multiplication turns the central coefficient calculation into exact algebra.

17.5 Coefficient recurrences

`FabiusSaddleCoefficientRecurrence.lean`, `FabiusSaddlePolynomialCoefficients.lean`, and `FabiusSaddleExpansionCoefficients.lean` organize the coefficients recursively. Each order depends only on finitely many cumulants and lower coefficients. The source avoids committing to a single giant closed form; a recurrence is more executable and easier to prove correct.

The recurrence is justified by differentiating or multiplying truncated formal series and comparing coefficients. Lean's finite-support polynomial representation ensures every coefficient sum is finite. Index constraints are handled with `Finset.antidiagonal` or explicit bounded ranges.

17.6 Uniform central remainders

Formal coefficients alone do not prove an asymptotic expansion. `FabiusSaddleCentralAllOrders.lean` bounds the difference between the exact central integrand and its truncated polynomial approximation uniformly on the growing central window.

The window grows slowly enough that the normalized Taylor remainder tends to zero at the requested inverse-phase rate, but fast enough that the omitted Gaussian tail is negligible. Choosing this window is a balancing act. The proof records explicit inequalities among its powers of λ, so later estimates reduce to comparisons of monomials.

17.7 All-order tails

`FabiusSaddleTailAllOrders.lean`, `GaussianPolynomialTailAllOrders.lean`, and `FabiusLambertAllOrderRemainder.lean` show that tail errors are smaller than every prescribed inverse power. Exponential decay is converted into polynomial decay using standard eventual inequalities:

$$e^{-c\lambda^{\alpha}} = O(\lambda^{-N}) \quad \text{for every fixed } N.$$

The source proves such comparisons with explicit thresholds and reuses them through Big-O transitivity.

17.8 Assembling the normalized mass expansion

`FabiusSaddleMassAllOrders.lean` combines central coefficient extraction and tail flatness. For each N, it proves

$$\mathcal{M}(x) - \sum_{j=0}^{N-1} \frac{c_j}{\lambda(x)^j} = O(\lambda(x)^{-N}).$$

The theorem is formulated through a structure or predicate such as `HasAsymptoticExpansion`, allowing generic lemmas for truncation, addition, multiplication, and logarithms.

17.9 From mass to logarithm

Since $c_0 = 1$, the normalized mass is eventually close to one. A formal logarithm of the coefficient series gives

$$\log \mathcal{M}(x) \sim \sum_{j\geq 1} \frac{d_j}{\lambda(x)^j}.$$

The analytic proof uses a Taylor theorem for $\log(1+u)$ with a uniform remainder on a small neighborhood. Positivity of the exact mass, inherited from the original integral representation, ensures the logarithm is well-defined.

17.10 Transfer to $F(x)$

`FabiusFullAsymptoticExpansion.lean` inserts the logarithmic mass expansion into the exact saddle reduction, adds the periodic and explicit main terms, and absorbs the exponentially small forward tail. The result is an all-order logarithmic expansion for $F(x)$ at zero.

The primary remainder is $O(\lambda^{-N})$. A weaker but simpler corollary uses $\lambda(x) \asymp -\log x$ to obtain

$$O((-\log x)^{-N}).$$

The line breaks in this display correspond to ordinary $-\log x$; they are separated here only to emphasize that positivity of the logarithmic scale is part of the eventual hypothesis.

17.11 First-order and Wikipedia-facing corollaries

Modules with names such as `FabiusWikipediaMain.lean` and `FabiusWikipediaExpansion.lean` package concise formulas suitable for comparison with public summaries. They separate what is correct at leading order from what is false at the claimed next order. The obstruction module demonstrates that a nonconstant dyadic-periodic correction cannot be absorbed into a vanishing error.

17.12 The proof-audit role of the asymptotic aggregator

`PaperFabiusAsymptotic.lean` is not just a theorem list. Its header records which informal claims are supported, which need corrected domains or periodic terms, and which are disproved. The formal development therefore functions as an audit trail:

- coarse log-squared behavior is validated;
- the exact delay relation is restricted to its true domain;
- a claimed overly small residual is refuted;
- the periodic Lambert-phase formula is proved independently through transforms and saddles; and
- arbitrary-order corrections are established at the exact phase.

Chapter 18

Legendre expansions and polynomial approximation

18.1 Why Legendre polynomials are a natural basis

The up-function is supported on $[-1, 1]$, is even, and is infinitely differentiable. Legendre polynomials are orthogonal on exactly this interval with respect to Lebesgue measure, so they provide a canonical spectral expansion:

$$\mathrm{up}(x) = \sum_{n \geq 0} a_n P_n(x).$$

Evenness forces every odd coefficient to vanish. The repository does not stop at a formal L^2 expansion: it evaluates the even coefficients exactly through moments, proves absolute and uniform convergence, translates the series back to the global Fabius function, and proves least-squares optimality of finite truncations.

18.2 Building the polynomial basis

`LegendrePolynomial.lean` develops the required Legendre facts in the normalization used by the project. The polynomials are defined through Rodrigues' formula or an equivalent derivative representation:

$$P_n(x) = \frac{1}{2^n n!} \frac{\mathrm{d}^n}{\mathrm{d}x^n}(x^2 - 1)^n.$$

The module proves degree, parity, endpoint values, differential equation, and derivative bounds.

A bespoke development is useful because later coefficient formulas require explicit monomial coefficients. A black-box orthogonal-polynomial theorem would provide orthogonality but not necessarily the exact finite sum compatible with the moment sequence.

18.3 Sturm–Liouville orthogonality

The Legendre differential equation is

$$\left((1 - x^2)P_n'(x)\right)' + n(n+1)P_n(x) = 0.$$

Multiplying the equation for P_n by P_m, the equation for P_m by P_n, subtracting, and integrating gives

$$(n(n+1) - m(m+1)) \int_{-1}^{1} P_n(x)P_m(x)\ \mathrm{d}x = 0.$$

Boundary terms vanish because of the factor $1 - x^2$. For $m \neq n$, the eigenvalues differ, proving orthogonality.

`FabiusLegendreSeries.lean` carries out this argument using interval integration by parts. The proof explicitly verifies differentiability of the polynomial expressions and endpoint vanishing. The eigenvalue inequality is discharged in natural arithmetic and cast to reals only at the final multiplication step.

18.4 Norms and pointwise bounds

The exact norm is

$$\int_{-1}^{1} P_n(x)^2\ \mathrm{d}x = \frac{2}{2n+1}.$$

The source derives it from Rodrigues' formula and repeated integration by parts, or from a recurrence plus normalization depending on the helper theorem used. This norm fixes the Fourier–Legendre coefficient:

$$a_n = \frac{2n+1}{2} \int_{-1}^{1} \mathrm{up}(x)P_n(x)\ \mathrm{d}x.$$

A bound $|P_n(x)| \leq 1$ on $[-1, 1]$ is proved from the differential equation and an extremum argument. This simple uniform bound is the last ingredient needed to convert absolute convergence of coefficients into uniform convergence of the function series.

18.5 Vanishing of odd coefficients

The parity theorem states

$$P_n(-x) = (-1)^n P_n(x).$$

Since up is even, the integrand $\mathrm{up}(x)P_n(x)$ is odd when n is odd. A symmetric-interval integral of an odd integrable function is zero. In Lean, the proof either invokes a generic odd-integral lemma or performs the substitution $x \mapsto -x$ and adds the two integral identities.

The resulting series is indexed naturally by P_{2n}. This is not only cosmetic: the exact coefficient formula then uses the even moment M_{2k}, for which the arithmetic normalization and denominator theory are available.

18.6 Exact coefficient evaluation

`FabiusLegendreCoefficients.lean` expands P_{2n} in monomials:

$$P_{2n}(x) = \sum_{k=0}^{n} c_{n,k} x^{2k}.$$

Finite linearity of the integral gives

$$a_{2n} = \frac{4n+1}{2} \sum_{k=0}^{n} c_{n,k} M_{2k}.$$

Every M_{2k} is the exact rational `moment` value, so a_{2n} is rational. The source also substitutes formulas expressing moments through inverse-power Fabius values, producing a coefficient formula entirely in terms of the exact dyadic table.

The proof is layered:

1. establish the explicit polynomial coefficient theorem;
2. turn polynomial evaluation into a finite sum;
3. commute the finite sum with the integral;
4. replace each monomial integral by the analytic moment;
5. identify the moment with the exact rational sequence; and
6. simplify casts and normalizations.

At no point is an infinite series exchanged with the integral during coefficient evaluation.

18.7 Generic convergence infrastructure

`LegendreSeriesConvergence.lean` develops a theorem for smooth functions whose repeated derivatives satisfy suitable endpoint or norm bounds. Integration by parts transfers derivatives from the function to the Legendre eigenstructure, yielding decay of coefficients faster than any fixed power of n.

For the Fabius up-function, compact support and smoothness supply these hypotheses to every order. Consequently the coefficient sequence is absolutely summable. Since $|P_n(x)| \le 1$, the Weierstrass M-test gives uniform convergence on $[-1, 1]$.

The generic theorem is separated from Fabius-specific coefficient arithmetic. This permits future reuse for other compactly supported smooth functions and keeps the main convergence proof independent of exact moment formulas.

18.8 Identifying the uniform limit

Orthogonality first shows that the partial sums are the L^2 projections of up. Absolute/uniform convergence produces a continuous limit S. To prove $S = \text{up}$, the source may use one of two equivalent routes:

- completeness of Legendre polynomials in $L^2[-1, 1]$, followed by equality of continuous representatives; or
- a generic convergence theorem already identifying the pointwise limit for sufficiently smooth functions.

The final theorem states a `HasSum` at each point, then upgrades to uniform convergence of the function series.

Endpoints are included. This matters because pointwise convergence theorems for orthogonal series often state only interior convergence. The absolute coefficient bound and $|P_n(\pm 1)| = 1$ handle endpoints directly.

18.9 Translated series for the global Fabius function

`FabiusTranslatedLegendreSeries.lean` transports the up-function expansion to an interval such as $[0, 2]$ where a translate agrees with the signed global Fabius function. The transformation replaces $P_{2n}(x)$ by $P_{2n}(x-1)$ or the corresponding affine normalization.

The module also expands translated Legendre polynomials into ordinary monomials. This produces an explicit power-polynomial series in the translated coordinate, with exact rational coefficients. Uniform convergence follows from the original series under composition with the affine map.

18.10 Least-squares optimality

`FabiusLegendreLeastSquares.lean` proves that the partial Legendre sum is the unique least-squares approximant among polynomials of the corresponding degree. Let

$$S_N = \sum_{n=0}^{N} a_n P_n.$$

For any polynomial p in the same finite-dimensional span,

$$\|\mathrm{up} - p\|_2^2 = \|\mathrm{up} - S_N\|_2^2 + \|S_N - p\|_2^2.$$

This Pythagorean identity follows because $\mathrm{up} - S_N$ is orthogonal to every basis polynomial in the span.

The formal proof represents the finite span either as a submodule of L^2-type functions or directly through polynomial coefficients. It proves orthogonality term by term and then expands the square of the norm. Uniqueness follows because equality of errors forces $\|S_N - p\|_2 = 0$, hence equality almost everywhere; polynomial continuity upgrades this to pointwise polynomial equality.

18.11 Computational significance

The exact coefficient theorem gives a certified way to produce rational polynomial approximants. The least-squares theorem certifies optimality in L^2, while uniform convergence certifies pointwise approximation. This is a rare combination:

- exact arithmetic coefficients;
- kernel-checked analytic convergence; and
- an optimality theorem for every finite truncation.

The evaluator and moment recurrences can be used to compute coefficients without numerical integration.

Chapter 19

The k-fold Thue–Morse draft audit

19.1 Purpose of the audit

`PaperKFoldThueMorse.lean` collects a separate line of work concerning k-fold Thue–Morse constructions and a related informal draft. Its role is both constructive and diagnostic: it proves exact identities and corrected approximation statements, but it also formalizes counterexamples to several literal claims in the draft.

This part of the repository illustrates a mature use of formalization. A failed proof is not treated as an obstacle to be papered over; the surrounding hypotheses are tested, false versions are refuted, and repaired versions are isolated.

19.2 Exact finite identities

The Thue–Morse modules prove prefix-sum cancellation, long zero runs of finite differences, convolution formulas, and generating-series identities. Repeated finite difference annihilates polynomials below a threshold degree, which explains the high-order cancellation of the sign sequence.

The formal proofs use induction on binary depth and repeated even/odd splitting. Convolution associativity is kept finite, so all sums are over `Finset.range` and can be reindexed explicitly. Generating polynomials turn convolution into multiplication, allowing coefficient comparison to replace nested sum manipulations.

19.3 Polygonal interpolation and corrected approximation

A piecewise-linear or polygonal interpolant converts finite coefficient data into a function. The draft's intended limit is recovered only after choosing a normalization and pointwise representative compatible with the mesh. The corrected theorem makes these choices explicit and proves convergence on the appropriate domain.

As in the step-approximation project, knot values require special treatment. Lean forces a decision about whether adjacent pieces are left-closed, right-closed, or averaged at the boundary. The corrected interpolation chooses a consistent convention and proves continuity directly.

19.4 Formal counterexamples

`DraftCounterexamples.lean` records counterexamples to claims about normalization, local and global errors, an unbounded maximum, and a missing linear term. A typical counterexample evaluates the proposed formula at the smallest nontrivial depth or at a symmetry point, where the exact recurrence reduces to rational arithmetic. `norm_num` or a short ring proof then contradicts the claimed equality or inequality.

Other false claims are asymptotic. The code constructs a sequence of inputs for which the normalized residual has a nonzero or divergent limit, using the already-established coarse Fabius asymptotic. The counterexample theorem is stated independently of prose, so its logical content is unambiguous.

19.5 Repairing the lower Lambert branch

The draft uses a lower branch of a Lambert-type inverse. The repository develops this branch with a correct domain, monotonicity, and inversion theorem. Endpoint behavior and branch separation are proved explicitly; no appeal is made to an ambiguous multivalued inverse.

These lemmas then support the corrected asymptotic comparisons in both the k-fold project and the principal Fabius saddle analysis.

19.6 Decay comparisons

`FabiusFlatness.lean` and `FabiusDecayComparison.lean` convert the log-squared law into two useful endpoint comparisons. For every $m > 0$,

$$F(x) = o(x^m) \qquad (x \downarrow 0),$$

so the function is smaller than every power. On the other hand, for every $c > 0$,

$$e^{-c/x} = o(F(x)),$$

so it is much larger than an $e^{-c/x}$ flat function.

Taking logarithms reduces these statements to comparing $(\log(1/x))^2$ with $\log(1/x)$ and with $1/x$. The code packages positivity and eventual monotonicity so the exponential comparison can use standard filter lemmas.

Chapter 20

Recurring proof-engineering patterns

20.1 Strong induction as the default arithmetic engine

Many sequences in the project depend on all smaller indices, not only on the predecessor. The idiomatic proof begins with strong induction:

```
refine Nat.strong_induction_on n ?_
intro n ih
-- ih : forall m < n, P m
```

A finite sum over `Finset.rangen` then supplies the inequality needed to invoke `ih`. It is often worth proving a helper lemma that converts membership in the range directly into the strict inequality expected by the hypothesis.

Strong induction also matches the highest-bit evaluator, whose recursive remainder is smaller but not necessarily one less. Using the same well-founded order for definition and correctness proof keeps the program and theorem structurally aligned.

20.2 Finite sums and index transport

The repository moves frequently among:

- `Finset.rangen`;
- subtype indices `Finn`;
- integer intervals;
- list positions; and
- antidiagonals describing coefficient convolution.

Rather than rewriting all indices inside a large expression, the code usually proves an equivalence between finite index types and invokes a reindexing theorem. A clean bijection proof has four parts: the forward map, inverse map, membership preservation, and left/right inverse equalities.

For even/odd splits, a specialized theorem is especially valuable:

$$\sum_{j<2N} f(j) = \sum_{k<N} f(2k) + \sum_{k<N} f(2k+1).$$

Once established, it drives Thue–Morse identities, convolution recurrences, and parity counts.

20.3 Casts as explicit interfaces

Exact theorems live in $\mathbb{N}$, $\mathbb{Z}$, or $\mathbb{Q}$; analytic theorems live in $\mathbb{R}$ or $\mathbb{C}$. Lean will insert canonical casts, but it will not silently prove that a finite sum commutes with the cast or that a natural exponent has the intended target type.

The code therefore proves and reuses lemmas of the forms

$$\left(\sum_{k\in s} q_k : \mathbb{Q}\right)\uparrow^{\mathbb{R}} = \sum_{k\in s}(q_k : \mathbb{R}),$$
$$(2^n : \mathbb{Q})\uparrow^{\mathbb{R}} = (2 : \mathbb{R})^n,$$
$$\left(\binom{n}{k} : \mathbb{N}\right)\uparrow^{\mathbb{Q}} = \binom{n}{k} \text{ interpreted in } \mathbb{Q}.$$

In actual Lean syntax the upward arrows are ordinary coercions. Tactics such as `norm_cast`, `exact_mod_cast`, and `push_cast` are used after the expression is put in a form they recognize.

Lean pattern

Do not ask `norm_cast` to solve a goal containing unreduced divisions, nested sums, and conditionals. First rewrite the exact recurrence, move casts through finite sums, simplify powers, and only then invoke cast normalization. The repository's helper lemmas reflect this staged approach.

20.4 Pointwise derivatives instead of raw `deriv`

A theorem of type `HasDerivAtf'x` carries both differentiability and the derivative value. It composes under addition, multiplication, affine maps, logarithms, and exponentials. By contrast, an equality involving `derivfx` often creates a side goal proving `DifferentiableAtf x` before it can be rewritten.

Accordingly, the calculus modules first prove `HasDerivAt` identities and derive `deriv` corollaries only when a source theorem is stated in that notation. Iterated derivatives are handled through `iteratedDeriv` or `ContDiff` lemmas with explicit order parameters.

20.5 Piecewise functions and boundary glue

The bounded function, fixed-point transform, splines, and endpoint representatives are all piecewise. The source follows a stable pattern:

1. define each branch as a total function;

2. prove branch continuity or differentiability;
3. prove equality of branch values at the boundary;
4. apply a piecewise glue theorem; and
5. derive simplified branch lemmas for later rewriting.

The simplified lemmas are often tagged for the simplifier only when their conditions are syntactically obvious. Overly aggressive `simp` tags on overlapping branches can create loops or choose an unintended normal form.

20.6 Local finiteness before infinite-sum algebra

For translate sums, the code first proves finite support at a point or on a neighborhood. Only then does it reindex or differentiate a `tsum`. This is safer than invoking general unconditional summability theorems on a series whose convergence is really trivial by support.

A typical theorem states that the summand is zero outside a finite integer interval. From this it derives summability, rewrites the `tsum` as a `Finset.sum`, performs the algebra, and converts back only if the public theorem uses `tsum` notation.

20.7 Integrability from support and bounds

The most common integrability proof has three ingredients:

- continuity or measurability of the integrand;
- compact support; and
- a uniform bound on that support.

The repository builds reusable support lemmas for F, up, their derivatives, and affine transforms. Polynomial or exponential multipliers remain bounded on a compact support, so moment and transform integrability follows from a generic compact-support theorem.

This style avoids global estimates that are stronger than needed. For instance, $x^n \operatorname{up}(x)$ is integrable not because x^n is globally bounded, but because the product vanishes outside $[-1, 1]$.

20.8 Interchanging sums, limits, derivatives, and integrals

Every interchange in the project is supported by a named convergence theorem:

- finite sums commute with everything by algebra;
- infinite sums commute with integrals under a summable integrable majorant;
- limits pass under integrals by dominated or bounded convergence;
- derivatives pass through series under local uniform convergence of derivative series; and
- contour limits use explicit uniform tail bounds.

The proof normally constructs the majorant first, proves its summability or integrability, and only then invokes the interchange theorem. This order makes failures local: if an informal series is not uniformly controlled, the missing hypothesis appears before the main identity is attempted.

20.9 Big-O, little-o, and eventual domains

Asymptotic modules use filters rather than epsilon notation. A theorem of the form

$$f(x) = O(g(x)) \quad (x \to 0^+)$$

is represented through `IsBigO` at a within-neighborhood filter. The proof often begins with

```
filter_upwards [eventually_pos, eventually_small] with x hxpos hxsmall
```

and establishes a pointwise norm inequality.

Eventual positivity is particularly important for logarithms, divisions, and Lambert phases. The source proves threshold lemmas once and then includes them in filter-upwards blocks. Transitivity of Big-O and equivalence under asymptotically equal scales are used heavily to transfer results from the exact phase $\lambda(x)$ to $-\log x$.

20.10 Polynomial identities

Exact finite identities are proved at the polynomial level whenever possible. The usual techniques are:

- extensionality by coefficients;
- evaluation at an arbitrary point followed by `ring`;
- induction using polynomial addition and multiplication; and
- degree bounds plus a roots theorem.

For translation-invariant Thue–Morse sums, a degree bound and finite-difference theorem often give a cleaner proof than expanding every binomial coefficient.

20.11 Choosing automation deliberately

The development uses automation, but proofs are structured so each tactic sees an appropriate goal:

`simp`
: unfolds branch lemmas, finite-range membership, and standard algebraic identities.

`norm_num`
: closes concrete rational and natural computations, especially counterexamples and base cases.

`ring` / `ring_nf`
: handles polynomial identities after divisions and casts have been normalized.

`linarith` / `nlinarith`
: combines real inequalities once transcendental terms have been abstracted as variables with known bounds.

`positivity`
: proves positivity of products, powers, factorials, exponentials, and denominators.

`omega`
: handles Presburger arithmetic for indices, ranges, and exponents.

`field_simp`
: clears rational or field denominators after nonzero hypotheses are established.

`aesop`
: resolves local structural goals, but is not used as a substitute for choosing the main induction or analytic theorem.

20.12 Public wrappers and internal strength

Theorem-producing modules tend to prove results in a strong reusable form: unrestricted numerators, arbitrary candidate functions, general coefficient rings, or explicit error constants. Paper-facing wrappers then add notation and specialize hypotheses.

When extending the library, search for the strongest internal theorem before reproving a source-shaped special case. Names ending in `_all`, `_global`, `_scalar`, `_correct`, `_transfer`, or `_of_isFabius` often signal the reusable layer.

20.13 No placeholders and explicit corrections

The repository states that the development builds without `sorry`. More importantly, corrected claims are not represented by axioms that merely assume the missing fact. When a source formula is false, the project either adds the necessary hypothesis, changes the object to the signed extension, weakens the error term, or proves a counterexample.

This makes the dependency graph epistemically meaningful. A theorem about the Fabius function ultimately reduces to mathlib, Lean's kernel, explicit definitions, and proved lemmas—not to a paper citation embedded as an axiom.

Chapter 21

How to read and extend the code

21.1 Four recommended reading paths

Different goals call for different entry points.

21.1.1 Path A: existence and uniqueness

Read, in order:

1. `Basic.lean`;
2. `Differential.lean`;
3. `DyadicClosedForm.lean`;
4. `DyadicAnalytic.lean`; and
5. `Existence.lean`.

This path explains the abstract specification, generic derivative consequences, dyadic rigidity, and fixed-point implementation.

21.1.2 Path B: exact evaluation

Read:

1. `Arithmetic.lean`;
2. `DyadicCorrectness.lean`;
3. `DyadicClosedForm.lean`;
4. `DyadicAnalytic.lean`; and
5. `GlobalDyadic.lean`.

Keep an editor evaluation buffer open and inspect the types of table, Horner, and wrapper correctness theorems.

21.1.3 Path C: the historical Rvachev theory

Read:

1. `GlobalExtension.lean`;
2. `FourierProduct.lean`;
3. `FourierAnalytic.lean`;
4. `OriginalUniqueness.lean`;
5. `ProbabilityRepresentation.lean`; and
6. `StepApproximationLimit.lean`.

This path emphasizes compact support, transform refinement, convolutions, and pointwise approximants.

21.1.4 Path D: sharp asymptotics

Read:

1. `FabiusLogScale.lean`;
2. `FabiusLogSquaredAsymptotic.lean`;
3. `NegativeLaplace.lean`;
4. `PeriodicCorrection.lean`;
5. `MellinBose.lean`;
6. `FabiusLambertPhase.lean`;
7. `BromwichSaddle.lean`;
8. `FabiusSharpExactReduction.lean`;
9. `FabiusSharpAsymptotic.lean`; and
10. `FabiusFullAsymptoticExpansion.lean`.

Use the specialized central/tail and all-order modules as dependencies are encountered.

21.2 Adding a new exact identity

Suppose a new conjectured formula concerns $F(a/2^n)$. A productive workflow is:

1. state and test the identity entirely in $\mathbb{Q}$ using `fabiusDyadicValue`;
2. normalize representations and determine whether the bounded or global extension is intended;
3. prove the finite arithmetic identity, preferably through existing closed-form or q-binomial lemmas;
4. only then transport it to the analytic function using dyadic correctness; and
5. add a paper-facing wrapper if the notation comes from an external source.

This approach keeps exploratory computation cheap and avoids starting with real analysis when the content is combinatorial.

21.3 Adding a new moment theorem

For a new moment recurrence:

1. derive the analytic recurrence in a generic `IsFabius` context;
2. define an exact rational sequence only if computation or arithmetic consequences justify it;
3. prove equality by strong induction;
4. design a division-free numerator recurrence before attempting parity or divisibility; and
5. separate reduced-denominator conclusions into a later module.

The existing normalized moment files are templates for common-denominator induction.

21.4 Adding an asymptotic correction

A proposed correction term should be attached to the exact saddle reduction rather than inferred from numerical fitting. The robust route is:

1. identify which cumulant or periodic Fourier coefficient contributes at the desired order;
2. extend the formal coefficient recurrence;
3. prove a uniform central remainder at that order;
4. prove the tail is smaller than the target scale;
5. update the normalized mass expansion; and
6. transfer through logarithm and the exact phase.

If the desired final formula is in elementary logarithms, derive it as a corollary of the exact-phase theorem using `FabiusLambertAllOrderSmallArgument.lean`.

21.5 Testing a suspected source error

Formalization is most efficient when the claim is stress-tested early.

- Check endpoint and smallest-index cases by exact evaluation.
- Verify domains of every differentiated functional equation.
- Compare bounded and signed-global versions under translation.
- For asymptotic claims, sample subsequences of fixed dyadic phase.
- For uniform errors, examine knots and support boundaries.
- For q-dependent finite rows, test nondyadic locations before claiming parameter independence.

A small counterexample theorem is often more informative than a long stalled proof attempt.

21.6 Maintaining dependency hygiene

Before importing the umbrella module into a new file, ask which interface is actually needed. An exact lemma should normally depend on `Arithmetic.lean` or a narrow q-binomial module, not on `Existence.lean`. A generic analytic lemma should depend on `Basic.lean` and the relevant calculus modules, not on the canonical instance unless the theorem truly uses it.

A useful check is to run the import-graph tooling included through mathlib's dependencies. Unexpected paths from asymptotic modules into paper aggregators or from arithmetic modules into analysis are signs that a helper theorem belongs in a lower layer.

21.7 Naming and theorem granularity

The repository's long module names encode the proof stage: `Raw`, `Scalar`, `ExactReduction`, `Transfer`, `Central`, `Tail`, `AllOrders`, and so on. New declarations should preserve this vocabulary where possible.

A theorem should expose one conceptual transition. Good boundaries include:

- closed formula equals Horner program;
- analytic recurrence equals exact recurrence;
- central integral equals Gaussian main term plus error;
- exact phase equals truncated elementary expansion plus error; and
- source statement follows from the stronger internal theorem.

Large proofs become navigable when these transitions have names.

21.8 A minimal downstream example

A downstream file that only needs exact evaluation and the analytic correctness theorem might begin as follows:

```
import FabiusFunction.DyadicAnalytic
import FabiusFunction.Existence

open Fabius

example (a n : Nat) :
    fabiusReal fabius (a / (2^n : Real)) =
      (fabiusDyadicValue a n : Real) := by
  -- apply the repository's dyadic correctness theorem,
  -- then discharge the chosen normalization side conditions
  simpa using fabius_dyadic_correct a n
```

The exact names and argument order should be confirmed in the editor; this schematic example illustrates the intended dependency level. Importing `FabiusFunction.lean` would also work,

but would hide which layer supplies the result.

21.9 What to trust when prose and code differ

The kernel-checked theorem type is the final authority. Module headers and README tables explain intent and source correspondence, while the proof term certifies the exact statement. When a paper formula differs from a Lean wrapper, inspect:

1. whether the paper silently assumes an index is positive;
2. whether a removable singularity has been totalized;
3. whether a global extension replaces a bounded function;
4. whether endpoint representatives differ only almost everywhere; and
5. whether an asymptotic periodic term has been omitted.

The repository usually records the reason in the aggregator header or a nearby correction theorem.

Chapter 22

Concluding perspective

The ProveIt development is best understood not as a very long proof of a single theorem, but as a small mathematical library centered on one function. It has several independent foundations:

- an exact arithmetic language of moments, binary signs, products, and dyadic evaluators;
- an abstract analytic interface capturing the Fabius differential and symmetry laws;
- a Banach fixed-point construction of the bounded solution;
- a compact-support/Fourier theory of the Rvachev twin; and
- a transform-and-saddle theory for the flat endpoint.

The strongest parts of the architecture are the bridges. The dyadic bridge identifies executable rationals with analytic values. The moment bridge identifies finite recurrences with integrals. The Fourier bridge identifies a refinement equation with an infinite product. The saddle bridge identifies that product with a corrected sharp asymptotic. The Legendre bridge turns exact moments into optimal polynomial approximants.

Formalization also changes the mathematical product. It forces endpoint conventions, domains of delay equations, transform normalizations, and bounded-versus-global distinctions to be stated. It exposes false residual estimates and missing terms, while often supplying a corrected theorem stronger than the source statement. The result is not merely a verification of known facts: it is a coherent, reusable account of how the facts fit together.

For a reader entering the source, the single most useful habit is to ask which side of the project a declaration belongs to: exact arithmetic, generic analysis, canonical construction, or a bridge. Once that question is answered, the long file list resolves into a small number of repeated proof patterns and a remarkably systematic dependency graph.

Appendix A

Detailed dependency map

A.1 Core spine

The shortest conceptual spine from definitions to the canonical function is:

$$\boxed{\texttt{Arithmetic.lean}} \longrightarrow \boxed{\texttt{Basic.lean}} \longrightarrow \boxed{\texttt{Differential.lean}}$$
$$\longrightarrow \boxed{\texttt{DyadicAnalytic.lean}} \longrightarrow \boxed{\texttt{Existence.lean}}.$$

This display omits two important side inputs. `DyadicClosedForm.lean` supplies the exact finite polynomial matched by `DyadicAnalytic.lean`, while `DyadicCorrectness.lean` supplies evaluator correctness and representation invariance. The resulting canonical function is then consumed by moment, Fourier, probability, global-extension, and asymptotic modules.

A.2 Arithmetic spine

The main arithmetic chain is:

$$\texttt{Arithmetic.lean} \longrightarrow \begin{cases} \texttt{NormalizedMoments.lean} \\ \texttt{NormalizedEvenMoments.lean} \\ \texttt{Parity.lean} \end{cases}$$
$$\longrightarrow \begin{cases} \texttt{HalfMomentDenominator.lean} \\ \texttt{TwoAdic.lean} \\ \texttt{DenominatorBound.lean} \end{cases} \longrightarrow \texttt{PaperStatements.lean}.$$

`AnalyticMoments.lean` and `ExactInversePower.lean` connect this chain to the canonical function before the source-facing wrappers are assembled.

A.3 Original-paper spine

The original Rvachev paper is represented by two parallel paths:

differential/refinement law → Fourier recurrence → sinc product → uniqueness,
finite elementary laws → convolutions → weak convergence → step approximants.

Their principal modules are `OriginalUniqueness.lean`, `FourierProduct.lean`, `ProbabilityRepresentation.lean`, `WeakConvergence.lean`, and `StepApproximationLimit.lean`. `OriginalPaperSupplement.lean` adds the remaining numbered theorems and prose claims.

A.4 Asymptotic spine

The sharp endpoint proof has the following high-level flow:

$$\begin{aligned}
&\texttt{AnalyticMoments.lean} \to \texttt{NegativeLaplace.lean} \to \begin{cases} \texttt{PeriodicCorrection.lean} \\ \texttt{MellinBose.lean} \end{cases} \\
&\qquad \to \texttt{BromwichSaddle.lean} \to \texttt{FabiusLambertPhase.lean} \\
&\qquad \to \begin{cases} \text{central modules} \\ \text{tail modules} \end{cases} \to \texttt{FabiusSharpExactReduction.lean} \\
&\to \texttt{FabiusSharpAsymptotic.lean} \to \texttt{FabiusFullAsymptoticExpansion.lean}.
\end{aligned}$$

The actual source graph is finer because the all-order proof separates formal coefficient algebra, small-argument Lambert expansion, Gaussian contraction, central uniformity, tail flatness, and transfer lemmas.

Appendix B

Paper theorem map

B.1 Arias de Reyna, arXiv:1702.05442

Source item	Lean declaration or family	Principal proof location
Theorem 1: structure and fold	`IsOriginalFabius`; `IsOriginalFabius.mk_of_derivative_law`; `IsFabius.isOriginalFabius_rvachevUp`	`OriginalCharacterization.lean`
Theorem 1: uniqueness and exact fold kernel	`isOriginalFabius_iff_eq_canonical`; `rvachevUp_eq_iff_eqOn_Iic_one`; `isFabius_iff_isOriginalFabius_rvachevUp_and_rightTail`; `isOriginalFabius_iff_existsUnique_isFabius`	`OriginalUniqueness.lean`
Lemma 1	finite-convolution probability convergence variants	`ProbabilityRepresentation.lean`, `WeakConvergence.lean`
Theorem 2	`stepApproximant_tendsto_rvachevUp` and endpoint-corrected variants	`StepApproximationLimit.lean`
Theorem 3	probability representation of the up-function	`ProbabilityRepresentation.lean`
Theorem 4	declarations grouped as `original_theorem_four_a`, `_b`, `_c`	`OriginalPaperSupplement.lean`, Fourier and differential helpers
Nonanalyticity	smoothness plus failure of analyticity	`OriginalPaperSupplement.lean`, differential and Taylor modules
Theorem 5	Poisson-summation identity	`PoissonSummation.lean`
Theorem 6	moment formulas	`AnalyticMoments.lean`, `OriginalPaperSupplement.lean`
Theorem 7	global rationality/dyadic statement	`GlobalDyadic.lean`, `OriginalPaperSupplement.lean`

B.2 Arias de Reyna, arXiv:1702.06487

Source item	Lean declaration or family	Principal proof location
Proposition 1	`proposition_one`	`PaperStatements.lean`; exact moment foundations
Proposition 2	`proposition_two` and real/removable-singularity version	`MomentPowerSeries.lean`, `PaperStatements.lean`
Proposition 3	`proposition_three`	analytic/exact moment recurrence modules
Proposition 4	`proposition_four`	moment and generating-function modules
Question 5	`reshetnikovQuestion`	recorded as a predicate/question in `PaperStatements.lean`
Proposition 6	`proposition_six`	exact arithmetic and dyadic modules
Theorem 7	`theorem_seven`	`DyadicClosedForm.lean`, `DyadicAnalytic.lean`
Proposition 8	`proposition_eight`	differential and dyadic Taylor modules
Theorem 9	`theorem_nine`, strengthened all-parameter form	`Paper06487Supplement.lean`, global/differential helpers
Lemma 1	`lemma_one`, with corrected order condition	`PaperStatements.lean` and arithmetic helpers
Proposition 10	`proposition_ten`	moment denominator normalization
Definition 12	`dyadicDenominator`	`PaperStatements.lean`
Theorem 13	`theorem_thirteen` and denominator bound	`DenominatorBound.lean`
Proposition 15	`proposition_fifteen`	normalized moments and divisibility
Conjecture 16	`conjecture_sixteen`	recorded without asserting a proof
Theorem 17	`theorem_seventeen`	parity and denominator modules
Proposition 18	`proposition_eighteen`	exact numerator/denominator analysis
Proposition 19	`proposition_nineteen`	exact arithmetic and parity
Theorem 20	`theorem_twenty`	`TwoAdic.lean`
Theorem 21	`theorem_twenty_one`	`TwoAdic.lean`; inverse-dyadic valuation
Proposition 22	initial and full variants of `proposition_twenty_two`	`Paper06487Supplement.lean`, dyadic/global modules

B.3 How to use the map

The declaration named after a source theorem is normally the best citation target in downstream formal work. The principal proof location is the best learning target. When the two differ, the wrapper is preserving source notation while the internal file proves a stronger or more reusable theorem.

Appendix C

Annotated module guide

The following guide groups the principal files by role. The development contains many fine-grained helper modules; the descriptions emphasize the transition each module contributes rather than every local lemma.

Module	Role in the development
Core interfaces and construction	
`Arithmetic.lean`	Exact natural/rational sequences, binary weight, Thue–Morse signs, product normalizations, inverse-power values, and executable dyadic evaluators.
`Basic.lean`	The unit-interval codomain, bounded-function type, abstract IsFabius specification, up-function, signed extension names, transforms, moments, and generating functions.
`Differential.lean`	Derivative identities for any IsFabius candidate, including iterated scale formulas and reflected branches.
`Existence.lean`	Banach fixed-point construction on symmetric bounded continuous functions, calculus bootstrap, canonical object, and abstract uniqueness packaging.
`ScaleTranslation.lean`	Reusable chain-rule formulas for affine rescaling and translation of Fabius and up-functions.
`GlobalExtension.lean`	Local finiteness, smoothness, unit-interval agreement, and translation laws of the signed global extension.
`GlobalDyadic.lean`	Exact dyadic identities for the signed extension with unrestricted and nonreduced representatives.
Dyadic formulas and verified computation	
`DyadicClosedForm.lean`	Closed finite Taylor formulas, highest-bit decompositions, and equivalence among exact dyadic expressions.
`DyadicCorrectness.lean`	Termination-facing invariants, table-prefix stability, Horner correctness, refinement invariance, and evaluator representation independence.
`DyadicAnalytic.lean`	Finite Taylor recurrence for an abstract IsFabius function and proof that analytic dyadic values equal the exact evaluator.
`ExactInversePower.lean`	Links inverse-power Fabius values with exact moments and table entries.

Module	Role in the development
`TaylorReduction.lean`	Finite Taylor-reduction identities used to move between dyadic cells.
`FabiusDyadicLogBounds.lean`	Explicit logarithmic bounds for inverse-dyadic values, used in endpoint asymptotics.
`DyadicSharpConditional.lean`	Conditional sharp statements at dyadic points under specified recurrence estimates.
`DyadicSharpDecomposition.lean`	Exact decomposition of dyadic logarithms into main and correction pieces.
`FabiusDyadicSharpCumulant.lean`	Cumulant organization specialized to inverse-dyadic asymptotics.
`FabiusDyadicSharpAsymptotic.lean`	Sharp asymptotic statements along the inverse-power sequence.
Moments, arithmetic, and denominators	
`AnalyticMoments.lean`	Compact-support integration, total mass, moment and half-moment recurrences, and identification with exact rational sequences.
`MomentPowerSeries.lean`	Entire series associated with moments and inverse-power values; bridge between coefficients and analytic generating functions.
`NormalizedMoments.lean`	Division-free normalization theorem for half moments using factorial and Mersenne products.
`NormalizedEvenMoments.lean`	Natural numerator and common-denominator formula for even centered moments.
`Parity.lean`	Lucas-theorem parity analysis and oddness of natural moment numerators.
`TwoAdic.lean`	Reduced numerator/denominator parity and exact two-adic valuation consequences.
`DenominatorBound.lean`	Common-denominator divisibility and explicit denominator bounds for source-facing arithmetic results.
`HalfMomentDenominator.lean`	Denominator control specialized to half moments and inverse-power values.
`BernoulliRecurrences.lean`	Translation of moment recurrences into Bernoulli-number identities used by the arithmetic paper.
`LaplaceMoments.lean`	Moment identities in the Laplace-transform normalization.
`LaplaceMomentBounds.lean`	Uniform factorial/exponential bounds needed for transform series and differentiation.
`FabiusRecurrenceSequence.lean`	Auxiliary exact recurrence sequence used in asymptotic and dyadic estimates.
Fourier, probability, and the first paper	
`FourierProduct.lean`	Convergence and continuity of the infinite sinc product and compatibility with finite products.
`FourierAnalytic.lean`	Fourier transform formula for the up-function, inversion consequences, and moment-series identification.
`OriginalCharacterization.lean`	Rvachev's original compact-support predicate, derived scale positivity, the forced scale and normalization, and the general fold from bounded Fabius solutions.

Module	Role in the development
`OriginalUniqueness.lean`	Fourier-refinement uniqueness, the exact kernel of the fold, and the equivalences with fixed candidates or unique bounded Fabius witnesses.
`ProbabilityRepresentation.lean`	Finite convolution laws, characteristic functions, and the probabilistic representation of the up-function.
`WeakConvergence.lean`	Convergence of finite probability measures against bounded continuous tests.
`StepMeasureBridge.lean`	Relates discrete/step approximants to their associated finite measures.
`EarlyMeasureBridge.lean`	Foundational bridge between early convolution approximants and measure formulations.
`StepApproximationLimit.lean`	Coefficient monotonicity, endpoint-corrected step functions, and pointwise convergence by an interval-average squeeze.
`EarlyApproximants.lean`	Definitions and elementary properties of the finite approximants used before the limiting theorem.
`PoissonSummation.lean`	Poisson-summation theorem in the repository's Fourier normalization.
`OriginalPaperSupplement.lean`	Remaining numbered and prose results of the first paper, including exact Taylor formulas and nonanalyticity.
`Paper05442.lean`	Aggregator and coverage audit for arXiv:1702.05442.
The arithmetic paper	
`PaperStatements.lean`	Source-numbered propositions, theorems, question, definition, and conjecture for arXiv:1702.06487.
`Paper06487Supplement.lean`	Prose-level and strengthened arithmetic-paper consequences, with corrected hypotheses and bounded/global distinctions.
`Paper06487.lean`	Compact aggregator for the complete arithmetic-paper formalization.
Finite q-binomial and Thue–Morse algebra	
`HalfQBinomial.lean`	Exact q-binomial coefficients at q=1/2, q-Pascal recurrence, Mersenne normalization, and a finite q-binomial theorem.
`FabiusRawQBinomialFormula.lean`	Raw coefficient-valued finite q-binomial identity before scalar or analytic specialization.
`FabiusRawQBinomialScalar.lean`	Scalar evaluation of the raw finite identity.
`FabiusQBinomialFormula.lean`	Principal exact Thue–Morse/q-binomial formula for unrestricted natural parameters.
`FabiusQBinomialScalarFormula.lean`	Scalar-valued form of the principal q-binomial theorem.
`FabiusDyadicQBinomialScalar.lean`	Specialization using exact dyadic Fabius values.
`FabiusQBinomialTaylor.lean`	Finite Taylor/coefficient analysis in the q parameter and exact remainder organization.
`FabiusGlobalQBinomialSeries.lean`	Convergent global series identified with the signed extension.
`FabiusBinaryReductionSeries.lean`	Binary-reduction telescoping series, including the necessary initial endpoint term.

Module	Role in the development
`FabiusParityPowerSeries.lean`	Power series whose coefficients reflect binary parity and Thue–Morse cancellation.
Discrete-limit family	
`FabiusUniformSpline.lean`	Centered finite Thue–Morse splines, support, knot compatibility, and uniform bounds.
`FabiusComplexShiftSpline.lean`	Taylor control of complex shifts of finite splines.
`FabiusDiscreteLimitToeplitz.lean`	Toeplitz-type averaging principle for normalized finite rows.
`FabiusDiscreteLimitComplexShift.lean`	Vanishing of complex-shift corrections under the limit scaling.
`FabiusDiscreteLimitIntegration.lean`	Final integration/telescoping step identifying the discrete limit with the Fabius extension.
Coarse endpoint asymptotics	
`FabiusLogScale.lean`	Defines the dyadic logarithmic coordinate and proves the domain-correct delay identity for the logarithm.
`FabiusLogSquaredAsymptotic.lean`	Quadratic logarithmic leading term, floor/ceiling transfer to real scale, and an explicit O(t log t) remainder.
`FabiusSmallArgumentScale.lean`	Filter and scale conversions between x tending to zero and the large logarithmic variable.
`FabiusFlatness.lean`	Proof that the Fabius function is smaller than every positive power at zero.
`FabiusDecayComparison.lean`	Comparison with exponential-flat functions such as exp(-c/x).
`FabiusLogMainDefect.lean`	Definition and first bounds for the defect after subtracting the quadratic logarithmic main term.
Negative Laplace, Mellin, and periodicity	
`NegativeLaplace.lean`	Infinite product and absolutely convergent logarithm, exact dilation equation, quadratic decomposition, periodic correction, and exponential tail.
`PeriodicCorrection.lean`	Continuity of the multiplicatively periodic correction by locally uniform convergence.
`PeriodicMean.lean`	Mean value and normalization of the period-one logarithmic correction.
`PeriodicRegularity.lean`	Differentiability and higher regularity of the periodic correction.
`PeriodicFourier.lean`	Fourier coefficients and nonconstancy of the periodic correction.
`MellinBose.lean`	Mellin transform of the Bose kernel and Gamma/zeta factorization.
`MellinFinitePart.lean`	Finite-part continuation after subtraction of singular Laurent terms.
`BoseFinitePartIntegral.lean`	Convergent integral representation for the regularized Bose/Mellin contribution.
`GammaSecondOrder.lean`	Second-order Gamma-function expansions and bounds used in saddle normalization.

Module	Role in the development
`LaplacePeriodicSecondOrder.lean`	Second-order transform asymptotics retaining the periodic correction.
Bromwich and Lambert saddle analysis	
`FabiusBromwichInput.lean`	Analytic and decay hypotheses needed to apply Bromwich inversion to the Fabius transform.
`BromwichSaddle.lean`	Vertical-contour inversion, saddle-line selection, and central/tail decomposition.
`FabiusLambertPhase.lean`	Exact implicit Lambert phase solving the saddle equation.
`FabiusLambertRates.lean`	Growth, comparison, and eventual positivity estimates for the exact phase.
`FabiusLambertSaddle.lean`	Substitution of the Lambert phase into the exponent and saddle identities.
`FabiusLambertDerivativeBounds.lean`	Uniform bounds for derivatives/cumulants at the saddle scale.
`FabiusLambertMinorArc.lean`	Modulus loss away from the central saddle window.
`FabiusLambertTailFlat.lean`	Exponential tail bounds upgraded to every inverse-power scale.
`FabiusSharpConstant.lean`	Exact normalization constant in the sharp asymptotic formula.
`FabiusSharpLambertMain.lean`	Assembly of the explicit Lambert-phase main term.
`FabiusSharpLambertTransfer.lean`	Transfer of estimates stated in saddle variables to the small-x filter.
`FabiusSharpExactReduction.lean`	Exact decomposition into explicit main term, normalized saddle mass, and forward-tail error.
`FabiusExplicitSharpTransfer.lean`	Conversion between exact sharp expressions and source-facing elementary forms.
`FabiusSharpAsymptoticTransfer.lean`	General transfer lemmas for logarithmic and exponential asymptotic equivalence.
`FabiusSharpAsymptotic.lean`	Correct sharp formula with periodic phase, first error bound, and asymptotic equivalence.
Central integral and all-order machinery	
`FabiusSaddleReduction.lean`	Algebraic reduction of the inversion integral to a normalized central weight.
`FabiusSaddleCentral.lean`	Leading Gaussian approximation in the central window.
`FabiusSaddleCentralLambert.lean`	Central estimate expressed at the exact Lambert saddle.
`FabiusSaddleCentralRadiusAsymptotics.lean`	Growth and compatibility conditions for the chosen central radius.
`FabiusSaddleTail.lean`	Coarse tail bound outside the central window.
`FabiusSaddleReferenceWeight.lean`	Normalized Gaussian reference integrals and identities.
`FabiusSaddleReferenceTail.lean`	Gaussian reference tail estimates.
`FabiusFirstSaddleCorrection.lean`	Explicit first non-Gaussian correction from cubic and quartic cumulants.

Module	Role in the development
`FabiusSaddleCoefficientRecurrence.lean`	Recursive definition and correctness of all-order expansion coefficients.
`FabiusSaddlePolynomialCoefficients.lean`	Polynomial coefficient extraction for the rescaled central exponent.
`FabiusSaddleExpansionCoefficients.lean`	Final scalar coefficients after Gaussian contraction.
`FabiusSaddleExponentAllOrders.lean`	Arbitrary-order expansion of the local exponent with uniform remainder.
`FabiusSaddleCentralAllOrders.lean`	Uniform central-integral expansion to every inverse phase order.
`FabiusSaddleTailAllOrders.lean`	Tail smaller than every requested inverse phase power.
`FabiusSaddleMassAllOrders.lean`	All-order expansion of the normalized saddle mass.
`GaussianPolynomialContraction.lean`	Linear functional sending even monomials to Gaussian moments and odd monomials to zero.
`GaussianPolynomialWholeIntegral.lean`	Exact whole-line Gaussian integrals of coefficient polynomials.
`GaussianPolynomialTail.lean`	Tail bounds for fixed Gaussian polynomials.
`GaussianPolynomialTailAllOrders.lean`	Uniform tail bounds compatible with arbitrary expansion order.
Lambert formal algebra and final expansion	
`FabiusLambertFormalLog.lean`	Formal logarithm/exponential algebra for truncated saddle series.
`FabiusLambertHigherExpansion.lean`	Higher explicit approximants to the exact Lambert phase.
`FabiusLambertAllOrderAlgebra.lean`	Coefficient identities for arbitrary-order Lambert substitution.
`FabiusLambertAllOrderRemainder.lean`	Analytic control of the remainder left by formal Lambert algebra.
`FabiusLambertAllOrderSmallArgument.lean`	Expansion of the exact phase in log(1/x) and log log(1/x) to arbitrary fixed order.
`FabiusFullAsymptoticExpansion.lean`	All-order expansion of log F at the exact phase and weakened elementary-log corollaries.
`FabiusWikipediaMain.lean`	Concise corrected main asymptotic in a public-summary format.
`FabiusWikipediaExpansion.lean`	Corrected higher expansion suitable for comparison with encyclopedic formulas.
`FabiusWikipediaObstruction.lean`	Proof that omission of the nonconstant periodic term invalidates a vanishing residual.
`FabiusQuotientExponentialMismatch.lean`	Rigorous mismatch between Fabius decay and a proposed quotient-of-exponentials fit.
`PaperFabiusAsymptotic.lean`	Aggregator and audit for the coarse, sharp, corrected, and all-order asymptotic program.
Legendre and approximation	

Module	Role in the development
`LegendrePolynomial.lean`	Rodrigues formula, parity, differential equation, monomial coefficients, norms, and endpoint data for Legendre polynomials.
`LegendreSeriesConvergence.lean`	Generic orthogonal-series convergence and coefficient-decay infrastructure for smooth functions.
`FabiusLegendreCoefficients.lean`	Exact even Legendre coefficients expressed through moments and inverse-dyadic values; odd coefficients vanish.
`FabiusLegendreSeries.lean`	Orthogonality, absolute/uniform convergence, and pointwise identification of the up-function series.
`FabiusTranslatedLegendreSeries.lean`	Affine translation to the signed Fabius function and monomial expansion of the translated series.
`FabiusLegendreLeastSquares.lean`	Pythagorean identity and unique least-squares optimality of finite Legendre truncations.
K-fold Thue–Morse audit and auxiliary asymptotics	
`ThueMorseApproximation.lean`	Corrected finite interpolation/approximation theorems for k-fold Thue–Morse data.
`ThueMorseBinomialLog.lean`	Binomial and logarithmic identities underlying the draft's proposed normalization.
`DraftCounterexamples.lean`	Exact and asymptotic counterexamples to false literal claims in the informal draft.
`StirlingAsymptotics.lean`	Factorial and binomial asymptotics needed for the k-fold scale.
`LowerLambertW.lean`	Correctly defined lower Lambert-type inverse branch, domain, monotonicity, and asymptotics.
`PaperKFoldThueMorse.lean`	Aggregator separating exact results, repaired statements, and formal counterexamples.
Top-level packaging	
`FabiusFunction.lean`	Umbrella import exposing the two papers, asymptotic audit, k-fold audit, negative-Laplace theory, and Legendre developments.

Remark C.1. Some helper files are intentionally absent from this table, and a few families may acquire additional leaves as the repository evolves. The umbrella module and the actual import graph remain the definitive inventory. Duplicate-looking module names usually mark distinct proof stages—for example, raw algebra versus scalar specialization, or a central estimate versus its all-order strengthening.

Appendix D

Corrections and critical distinctions

Topic	Informal hazard	Formal treatment
Bounded versus global F	A translation or dyadic formula is written for a clamped function on all of $\mathbb{R}$.	Use `extendedFabius` for the signed global statement; prove agreement with the bounded function on the unit interval.
Original finite convolution	A limiting convolution object is used where a finite-stage identity is required.	State the convolution theorem for each finite row, then pass to the limit through weak convergence.
Step endpoints	Closed interval indicators count a shared endpoint twice.	Define a half-weight endpoint representative when the theorem is pointwise; retain ordinary indicators for almost-everywhere/integral statements.
Fourier equation factor	A transform variable or scaling factor is dropped during differentiation/dilation.	Derive the identity from mathlib's Fourier derivative and affine-map theorems under a fixed convention.
Positive index	A denominator, derivative order, or truncated product is used at $n = 0$ although the formula assumes $n > 0$.	Add the explicit order hypothesis and prove the zero case separately when meaningful.
Removable singularity	A quotient such as $(e^z - 1)/z$ is treated as globally defined.	Define a total function with the limiting value at zero and prove a global power-series theorem.
Dyadic representation	A formula is proved only for a reduced numerator even though later sums produce unreduced fractions.	Prove refinement invariance and formulate global correctness for arbitrary representatives.
Taylor series	Exact finite Taylor truncation is mistaken for local analyticity.	Use a finite-order Taylor theorem with vanishing remainder; separately prove the analytic-at predicate fails.

Topic	Informal hazard	Formal treatment
Logarithmic delay equation	A branch-dependent derivative law is differentiated without a domain.	State the transformed identity only for $t \geq 1$ and use eventual filters thereafter.
Periodic asymptotic term	A bounded log-periodic correction is absorbed into an $o(1)$ or inverse-log error.	Prove the correction is nonconstant and exhibit phase subsequences with different residual limits.
Binary-reduction series	The outer sum begins after the endpoint contribution.	Start at $m = 0$ or include the missing boundary term explicitly so finite telescoping is exact.
Finite q dependence	Independence of the limiting value is promoted to independence of every approximant.	Preserve the complex q parameter at finite depth; prove its effect tends to zero through complex-shift estimates.
Saddle expansion composition	A truncated Lambert expansion is substituted to all orders without a composition remainder proof.	State the primary all-order theorem at the exact phase and prove elementary-log expansions separately with implicit-function error bounds.
Quotient exponential fit	Numerical agreement over a finite range is treated as asymptotic equivalence.	Compare logarithmic scales and prove the proposed quotient has the wrong endpoint decay.
K-fold normalization	A draft normalization or global error is assumed from plots or low-depth data.	Evaluate exact small cases and formalize counterexamples before proving a repaired statement.

D.1 What these corrections teach

The corrections fall into four recurring classes. Domain errors arise when a local functional equation is used globally. Representation errors arise when a bounded object is substituted for a signed extension or an almost-everywhere representative is substituted in a pointwise claim. Normalization errors arise from Fourier, convolution, and dyadic scaling conventions. Asymptotic errors arise when a bounded oscillation is mistaken for a small remainder or when a formal series is composed without analytic control.

Lean detects each class differently. A domain error appears as an unprovable interval side condition. A representation error appears as a concrete endpoint contradiction. A normalization error leaves an unmatched scalar factor. An asymptotic error usually survives local algebra but fails at the limit theorem; the repository then proves a negative result by constructing a subsequence or comparing scales.

Appendix E

Lean and mathematical glossary

Term or identifier	Meaning in this development
`UnitInterval`	The subtype $\{x : \mathbb{R} \mid 0 \le x \le 1\}$, used as the codomain of the bounded function.
`BoundedFabius`	A real-input function whose values carry the unit-interval bound in their type.
`fabiusReal`	Real-valued projection of a subtype-valued bounded function, suitable for calculus and integration.
`IsFabius`	Abstract specification combining tail values, smoothness, symmetry, and the left-half derivative law.
`fabius` / `fabius_spec`	The canonical constructed object and the theorem that it satisfies the specification.
`rvachevUp`	Compactly supported even twin used in Fourier and probability arguments.
`extendedFabius`	Signed globally defined extension assembled from Thue–Morse-weighted translates of the up-function.
`HasDerivAt`	Pointwise derivative proposition carrying differentiability and the derivative value together.
`ContDiff`	Smoothness predicate to finite or infinite order.
`tsum`	Infinite sum over a type; translate sums are proved locally finite before this is manipulated.
`HasSum`	Proposition that a series has a specified sum; often more convenient than rewriting a `tsum`.
`Finset.rangen`	Finite set $\{0, \dots, n-1\}$, the dominant indexing object for recurrences and finite products.
`Nat.strong_induction_on`	Well-founded induction giving hypotheses for every smaller natural index.
`IsBigO`, `IsLittleO`	Filter-based asymptotic bounds and negligibility.
`atTop`	Filter describing a variable tending to positive infinity.
`nhdsWithin0(Set.Ioi 0)`	Right-hand approach to zero, the endpoint filter used for $x \downarrow 0$.
`eventually`	A property holding on some set belonging to a filter; used to encode large-variable thresholds.
`Summable`	Absolute or unconditional summability in the relevant normed additive group.

Term or identifier	Meaning in this development
`intervalIntegral`	Oriented integral over real endpoints, useful for FTC and integration by parts.
`MeasureTheory.Integrable`	Whole-space integrability; usually proved from compact support and continuity.
`SchwartzMap`	Smooth rapidly decreasing function type used for Fourier injectivity and Poisson summation.
`fabiusDyadicValue`	Executable exact rational evaluator for a dyadic input representation.
`fabiusDyadicUnitAux`	Well-founded highest-bit recursion at the core of unit-interval evaluation.
`halfMoment` / `moment`	Exact rational recurrence sequences later identified with analytic integrals.
`halfMomentNumerator` / `momentNumerator`	Division-free natural recurrences used for parity and denominator proofs.
`binaryWeight`	Number of one-bits of a natural number.
`thueMorseSign`	Sign $(-1)^{s_2(n)}$.
`halfQBinomial`	Exact q-binomial coefficient specialized to $q = 1/2$.
`field_simp`	Tactic for clearing field denominators after nonvanishing hypotheses have been supplied.
`norm_cast` / `push_cast`	Tactics normalizing canonical casts across algebraic operations.
`ring_nf`	Polynomial normalizer used after divisions, casts, and finite sums are brought to algebraic normal form.
`filter_upwards`	Tactic for collecting eventual hypotheses and proving a pointwise eventual conclusion.

Appendix F

Example exploration snippets

The following snippets are intentionally schematic where theorem argument order may change. They show the style of interaction rather than replacing editor inspection of the current declarations.

F.1 Inspect the public objects

```
import FabiusFunction

open Fabius

#check UnitInterval
#check BoundedFabius
#check IsFabius
#check fabius
#check fabius_spec
#check rvachevUp
#check extendedFabius
```

F.2 Evaluate exact dyadic data

```
import FabiusFunction.Arithmetic

open Fabius

#eval fabiusInversePowTwoTable 8
#eval fabiusDyadicValue 1 1
#eval fabiusDyadicValue 5 4
#eval extendedFabiusDyadicValue 13 5
```

The returned objects are exact rationals. To evaluate a rational input through the recognition layer, inspect the declarations near `IsDyadicRational`, `dyadicExponent?`, and the evaluator wrappers in `Arithmetic.lean`.

F.3 Work generically under the specification

```
import FabiusFunction.Differential

open Fabius

variable (F : BoundedFabius) (hF : IsFabius F)

#check hF.symmetry_all
-- Search within Differential.lean for the iterated derivative
-- theorem whose interval hypotheses fit the desired dyadic cell.
```

The generic style is useful for proving a new consequence once and applying it both to the canonical function and to any future equivalent construction.

F.4 Find source-facing arithmetic theorems

```
import FabiusFunction.Paper06487

open Fabius

#check proposition_one
#check theorem_seven
#check theorem_twenty
#check theorem_twenty_one
#check conjecture_sixteen
```

A declaration representing a conjecture is a definition or proposition describing the assertion, not a proof of it.

F.5 Inspect asymptotic interfaces

```
import FabiusFunction.PaperFabiusAsymptotic

open Fabius

-- Use #check on the named main, obstruction, and full-expansion
-- theorems, then follow their exact-phase helper declarations.
#check Fabius.fabiusLogScale
```

Because filter statements can be long, editor hover information is often more useful than printing the entire type in a terminal.

F.6 Search tactics

Within an editor, useful text searches include:

```
IsFabius -> HasDerivAt
```

```
fabiusDyadicValue -> Real
rvachevUp -> Fourier
negativeLaplaceLog
HasAsymptoticExpansion
Legendre -> HasSum
```

Searching by the transition one wants—rather than by a source theorem number—usually finds the strong internal lemma.

Appendix G

Bibliographic and repository notes

Bibliography

[1] Vladimir Reshetnikov, *ProveIt: Analysis/FabiusFunction*, GitHub repository, source tree and README inspected 24 August 2026. https://github.com/VladimirReshetnikov/ProveIt/tree/main/Analysis/FabiusFunction

[2] Juan Arias de Reyna, *An infinitely differentiable function with compact support: definition and properties*, arXiv:1702.05442 (2017). https://arxiv.org/abs/1702.05442

[3] Juan Arias de Reyna, *Arithmetic of the Fabius function*, arXiv:1702.06487, version 3 (2017). https://arxiv.org/abs/1702.06487

[4] Leonardo de Moura and Sebastian Ullrich, *The Lean 4 Theorem Prover and Programming Language*, Automated Deduction – CADE 28, 2021.

[5] The mathlib community, *The Lean mathematical library*, CPP 2020 and continuously maintained Lean 4 library. https://github.com/leanprover-community/mathlib4

Document metadata

Title	The Fabius Function in Lean: A Guided Walkthrough of the ProveIt Formalization
Source snapshot	`main` branch inspected 24 August 2026
Toolchain recorded by source	Lean 4.32.0; repository-pinned mathlib revision
Primary path	`Analysis/FabiusFunction/Lean/`
Umbrella module	`FabiusFunction.lean`
Document purpose	Human-readable structural and proof-oriented guide to the Lean development